PIPELINE

Las funciones y clases que menciona el pipeline de FonoKids, en el orden de sus cuatro fases, con cada línea explicada.

Cómo leerlo: a la izquierda está el código tal como está hoy en el proyecto, con su número de línea; a la derecha, qué hace esa línea (o ese grupo de líneas que forman una sola instrucción). Los nombres de funciones y clases están en inglés. Si editas los archivos, los números de línea se corren.

Dos cosas a tener en cuenta: (1) Los pasos de limpieza de los puntos (`normalize_lip_landmarks`, `is_outlier`, `fill_missing_frames`, `smooth_positions`, `add_dynamics`) están explicados sobre `scripts/extraer_landmarks_npy.py`, que arma el dataset; en el servidor son el mismo código, en `server/main.py` líneas 149, 190, 196, 232 y 240. (2) La Fase 4 (la app en React) no lleva código línea por línea: sus archivos principales son `web/src/screens/HomeScreen.jsx` (graba y dibuja la cara), `web/src/api.js` (habla con el servidor) y `web/src/ejercicios.js`.

Contenido (43 partes, 1266 líneas explicadas)

Fase 1 · Captura y preparación (navegador y servidor)

1. encodeWav y createAudioRecorder (el navegador graba el audio) (audioRecorder.js:7-93)
2. build_hrnet_detector (crea el detector HRNet) (main.py:89-102)
3. detect_landmarks_batch (los 68 puntos de cada frame) (main.py:105-146)
4. normalize_lip_landmarks (extraer_landmarks_npy.py:230-249)
5. normalize_face_landmarks (los 68 puntos, medidos con los ojos) (main.py:171-184)
6. is_outlier (y su constante) (extraer_landmarks_npy.py:252-258)
7. fill_missing_frames (extraer_landmarks_npy.py:261-291)
8. smooth_positions (y su constante) (extraer_landmarks_npy.py:294-302)
9. add_dynamics (extraer_landmarks_npy.py:305-312)
10. preprocess_sequence (deja la secuencia lista para el modelo) (main.py:536-554)
11. process_clip (extraer_landmarks_npy.py:315-351)

Fase 2 · Datos y entrenamiento (PyTorch)

12. LipReadingDataset (cómo se leen los .npy para entrenar) (lip_reading_dataset.py:20-96)
13. build_dataloaders (separar entrenamiento y validación) (lip_reading_dataset.py:99-130)
14. PositionalEncoding (train_landmarks_transformer.py:29-40)
15. TCNBlock (train_landmarks_transformer.py:43-62)
16. TCNFeatureExtractor (train_landmarks_transformer.py:65-76)
17. ConformerFeedForward (train_landmarks_transformer.py:79-93)
18. ConformerConvModule (train_landmarks_transformer.py:96-128)
19. ConformerBlock (train_landmarks_transformer.py:131-161)
20. ConformerEncoder (train_landmarks_transformer.py:164-176)
21. LipReadingConformer (el modelo de video) (train_landmarks_transformer.py:179-214)
22. masked_mean_pool y make_padding_mask (train_landmarks_transformer.py:217-229)
23. run_epoch (una vuelta por todos los datos) (train_landmarks_transformer.py:272-302)
24. train (el modelo de video) (train_landmarks_transformer.py:407-489)
25. load_model (carga wav2vec2) (audio_pronunciation.py:129-152)
26. load_audio_16k (audio_pronunciation.py:155-161)
27. extract_embedding (y su constante) (audio_pronunciation.py:164-173)
28. AudioClassifierHead (el modelo de audio) (train_audio_classifier.py:95-107)
29. EarlyStopping (train_audio_classifier.py:110-126)
30. Constantes del entrenamiento de audio (train_audio_classifier.py:19-26)

31. main (entrenar el modelo de audio) (train_audio_classifier.py:168-284)

Fase 3 · Análisis de pronunciación (servidor FastAPI)

32. _phoneme_to_token_ids y _greedy_ctc_decode (fonemas ↔ números) (audio_pronunciation.py:176-195)
33. _distancia_edicion y _align_ops (comparar lo esperado con lo oído) (audio_pronunciation.py:205-253)
34. diagnose_tipo_error (qué tipo de error fue) (audio_pronunciation.py:256-293)
35. Constantes del GOP (audio_pronunciation.py:14-16)
36. score_pronunciation (audio_pronunciation.py:296-361)
37. Constantes del GOP para frases (audio_pronunciation.py:364-366)
38. score_word_in_sentence (audio_pronunciation.py:385-498)
39. Umbrales de la decisión (main.py:57-61)
40. _classify_audio (main.py:891-919)
41. _run_prediction_pipeline: el video, paso a paso (main.py:996-1112)
42. La decisión final: audio + video (dentro de _run_prediction_pipeline) (main.py:1113-1183)
43. predict (el endpoint /predict) (main.py:1212-1262)

Fase 1 · Captura y preparación (navegador y servidor)

1. encodeWav y createAudioRecorder (el navegador graba el audio)

web/src/audioRecorder.js:7-93

El servidor necesita un WAV de verdad (16 bits, un canal). El navegador no lo da hecho, así que esta parte junta las muestras de audio y arma el archivo WAV a mano, byte por byte.

`7 function encodeWav(chunks, sampleRate) {`	encodeWav recibe los trozos de audio y la frecuencia de muestreo, y devuelve el archivo WAV.
`8 let totalLength = 0;` `9 for (const c of chunks) totalLength += c.length;`	Suma cuántas muestras hay entre todos los trozos.
`11 const pcm16 = new Int16Array(totalLength);`	Crea un arreglo de enteros de 16 bits donde caben todas las muestras.
`12 let offset = 0;`	Posición donde se va escribiendo la siguiente muestra.

`13 for (const chunk of chunks) {`
`14 for (let i = 0; i < chunk.length; i++) {`
`15 const s = Math.max(-1, Math.min(1, chunk[i]));`
`16 pcm16[offset++] = s < 0 ? s * 0x8000 : s * 0x7fff;`
`17 }`
`18 }`
	Recorre cada muestra: la limita entre -1 y 1 y la convierte a entero de 16 bits (las negativas se multiplican por 32768 y las positivas por 32767).

`20 const buffer = new ArrayBuffer(44 + pcm16.length * 2);` `21 const view = new DataView(buffer);`	Reserva el archivo: 44 bytes de cabecera más 2 bytes por muestra, y prepara un DataView para escribir byte a byte.

`22 const writeStr = (pos, str) => {`
`23 for (let i = 0; i < str.length; i++) view.setUint8(pos + i, str.charCodeAt(i));`
`24 };`
	writeStr escribe un texto (como «RIFF») letra por letra dentro de la cabecera.

`25 writeStr(0, "RIFF");`	Marca de inicio del archivo: «RIFF».
`26 view.setUint32(4, 36 + pcm16.length * 2, true);`	Tamaño del resto del archivo.

`27 writeStr(8, "WAVE");`
`28 writeStr(12, "fmt ");`
`29 view.setUint32(16, 16, true);`
	«WAVE», el bloque «fmt » y su tamaño (16 bytes).

`30 view.setUint16(20, 1, true); // PCM`	Formato 1 = PCM, sin comprimir.
`31 view.setUint16(22, 1, true); // mono`	1 canal (mono).
`32 view.setUint32(24, sampleRate, true);`	Muestras por segundo (la del navegador).
`33 view.setUint32(28, sampleRate * 2, true);`	Bytes por segundo: muestras por segundo por 2.
`34 view.setUint16(32, 2, true);`	Bytes por muestra: 2.
`35 view.setUint16(34, 16, true);`	Bits por muestra: 16.
`36 writeStr(36, "data");` `37 view.setUint32(40, pcm16.length * 2, true);`	Bloque «data» y cuántos bytes de audio trae.
`38 new Int16Array(buffer, 44).set(pcm16);`	Copia las muestras justo después de la cabecera.
`40 return new Blob([buffer], { type: "audio/wav" });` `41 }`	Devuelve el archivo como Blob de tipo audio/wav.

`43 /// Crea un grabador independiente (sin estado de React) -- start(stream)`
`44 /// arranca a juntar PCM crudo del track de audio del stream, stop()`
`45 /// corta y devuelve el Blob .wav ya armado (o null si no había audio).`
	Comentario: este grabador no depende de React; start() empieza a juntar audio y stop() devuelve el WAV.

`46 export function createAudioRecorder() {`	createAudioRecorder crea un grabador nuevo.

47 let ctx = null; 48 let source = null; 49 let processor = null; 50 let chunks = [];	Variables internas: el contexto de audio, la fuente, el procesador y la lista de trozos grabados.
52 function start(stream) { 53 const audioTrack = stream?.getAudioTracks?.()[0]; 54 if (!audioTrack) return false; 55 chunks = [];	start(stream): toma la pista de audio del micrófono; si no hay, devuelve false; si hay, vacía la lista de trozos.
56 try { 57 const AudioCtx = window.AudioContext || window.webkitAudioContext; 58 ctx = new AudioCtx(); 59 source = ctx.createMediaStreamSource(new MediaStream([audioTrack]));	Crea el AudioContext (con la versión de Safari si hace falta) y una fuente que lee solo la pista de audio.
60 processor = ctx.createScriptProcessor(4096, 1, 1);	Crea un procesador que recibe el audio en bloques de 4096 muestras.
61 processor.onaudioprocess = (e) => { 62 chunks.push(new Float32Array(e.inputBuffer.getChannelData(0))); 63 };	Cada vez que llega un bloque, copia sus muestras (canal 0) a la lista.
64 source.connect(processor); 65 processor.connect(ctx.destination);	Conecta fuente, procesador y salida; sin esto el procesador no corre.
66 return true;	Devuelve true: la grabación empezó.
67 } catch (e) { 68 console.warn("No se pudo iniciar la captura de audio:", e); 69 return false; 70 }	Si algo falla avisa en la consola y devuelve false.
71 }	Cierra start.
73 function stop() { 74 if (!ctx) return null; 75 const sampleRate = ctx.sampleRate;	stop(): si no hay contexto, devuelve null; si hay, guarda la frecuencia de muestreo.
76 try { 77 processor?.disconnect(); 78 source?.disconnect(); 79 ctx.close();	Desconecta el procesador y la fuente y cierra el contexto.
80 } catch { 81 // no crítico 82 }	Si falla al cerrar no pasa nada grave.
83 const finalChunks = chunks; 84 ctx = null; 85 source = null; 86 processor = null; 87 chunks = [];	Guarda los trozos grabados y deja todo vacío para la próxima vez.
88 if (finalChunks.length === 0) return null;	Si no se grabó nada, devuelve null.
89 return encodeWav(finalChunks, sampleRate); 90 }	Arma y devuelve el WAV con encodeWav.
92 return { start, stop }; 93 }	Devuelve las dos funciones, start y stop.

2. build_hrnet_detector (crea el detector HRNet)

server/main.py:89-102

Crea el modelo que ubica los 68 puntos de la cara (HRNet/FAN). Se crea una sola vez al arrancar el servidor.

89 def build_hrnet_detector():	Define la función.
90 import face_alignment 91 import torch._dynamo	Importa face_alignment (el paquete que trae el modelo ya entrenado) y la herramienta de PyTorch que compila código.

93 torch._dynamo.config.suppress_errors = True	Si el compilador falla, que siga sin detener todo.
95 if DEVICE == "cpu": 96 print("[AVISO] No hay GPU disponible -- HRNet/FAN en CPU va a ser MUY lento.")	Si no hay GPU, avisa que va a ser muy lento.
98 detector = face_alignment.FaceAlignment(99 face_alignment.LandmarksType.TWO_D, device=DEVICE, flip_input=False, compile=False, 100)	Crea el detector de puntos en 2D, en la GPU si hay, sin compilar y sin repetir la imagen volteada.
101 print(f">>> Detector HRNet/FAN corriendo en {DEVICE.upper()}. <<<")	Avisa en qué dispositivo quedó corriendo.
102 return detector	Devuelve el detector.

3. detect_landmarks_batch (los 68 puntos de cada frame)

server/main.py:105-146

Recibe todos los frames del video y devuelve, para cada uno, los 68 puntos de la cara (o None si no hay cara). Trabaja de a 4 frames para ir más rápido.

105 def detect_landmarks_batch(detector, frames_bgr):	Define la función: recibe el detector y la lista de frames.
106 if not frames_bgr: 107 return []	Si no hay frames, devuelve una lista vacía.
109 frames_rgb = [cv2.cvtColor(f, cv2.COLOR_BGR2RGB) for f in frames_bgr]	OpenCV entrega los colores como BGR; el detector los quiere como RGB.
110 device = detector.face_detector.device 111 chunk_size = HRNET_BATCH_CHUNK_SIZE	Toma el dispositivo del detector y el tamaño del lote (4 frames).
112 detected_faces = []	Aquí se juntan las caras encontradas.
113 try:	Empieza un bloque protegido: si el modo por lotes falla, hay otro plan.
114 for start in range(0, len(frames_rgb), chunk_size): 115 chunk = frames_rgb[start:start + chunk_size]	Recorre los frames de a 4.
116 batch = torch.stack([117 torch.from_numpy(np.ascontiguousarray(f)).permute(2, 0, 1) for f in chunk 118]).to(device)	Convierte esos 4 frames en un solo tensor (canales primero) y lo manda a la GPU.
119 try: 120 chunk_faces = detector.face_detector.detect_from_batch(batch)	Busca las caras de los 4 frames a la vez.
121 except torch.cuda.OutOfMemoryError:	Si la GPU se queda sin memoria...
122 del batch 123 print(f"[AVISO] Sub-lote de {len(chunk)} frames sin memoria, reintentando de a un frame.") 124 chunk_faces = []	libera el lote, avisa y prepara una lista vacía...
125 for frame in chunk: 126 single = torch.from_numpy(np.ascontiguousarray(frame)).permute(2, 0, 1).unsqueeze(0).to(device) 127 chunk_faces.extend(detector.face_detector.detect_from_batch(single)) 128 del single	y repite de a un frame: cada uno se manda solo a la GPU, se buscan sus caras y se libera.
129 detected_faces.extend(chunk_faces) 130 del batch	Guarda las caras del lote y libera la memoria.
131 except Exception as e: 132 print(f"[AVISO] Deteccion en batch fallo ({e}), usando frame por frame.") 133 detected_faces = None	Si todo el modo por lotes falla, avisa y deja detected_faces en None para trabajar frame por frame.

135 resultados = []	Lista donde irán los puntos de cada frame.
136 for i, frame_rgb in enumerate(frames_rgb):	Recorre los frames uno por uno.
137 if detected_faces is None: 138 preds = detector.get_landmarks_from_image(frame_rgb)	Si se está en modo frame por frame, el detector busca cara y puntos por su cuenta.
139 else: 140 faces = detected_faces[i] if i < len(detected_faces) else [] 141 if not len(faces): 142 resultados.append(None) 143 continue 144 preds = detector.get_landmarks_from_image(frame_rgb, detected_faces=faces)	Si no, toma las caras ya encontradas de ese frame; si no hay ninguna, anota None y sigue; si hay, calcula los puntos usando esa cara.
145 resultados.append(preds[0].astype(np.float32) if preds else None) 146 return resultados	Guarda los 68 puntos del primer rostro (o None) y al final devuelve la lista completa.

4. normalize_lip_landmarks

scripts/extraer_landmarks_npy.py:230-249

Convierte los puntos de la boca, que vienen en píxeles, a coordenadas que no dependen de dónde está la cara en el video, ni de qué tan cerca está de la cámara, ni de cuánto está inclinada. Es el primer paso antes del modelo de video.

230 def normalize_lip_landmarks(landmarks_px):	Define la función. Recibe landmarks_px: los 68 puntos de la cara de un frame, en píxeles.
231 right_corner = landmarks_px[MOUTH_RIGHT_CORNER] 232 left_corner = landmarks_px[MOUTH_LEFT_CORNER]	Toma las dos comisuras (esquinas) de la boca: el punto 48 (derecha) y el 54 (izquierda).
234 mouth_center = (right_corner + left_corner) / 2.0	Calcula el centro de la boca: el punto medio entre las dos comisuras.
235 mouth_vector = left_corner - right_corner	Calcula el vector que va de una comisura a la otra (su dirección y su largo).
236 scale = np.linalg.norm(mouth_vector)	scale es el ancho de la boca en píxeles (el largo de ese vector). Sirve para quitar el efecto de qué tan cerca está la cara de la cámara.
237 if scale < 1e-3: 238 return None	Si las comisuras casi se superponen (ancho menor a 0,001), algo salió mal en la detección: devuelve None para que ese frame se rellene después.
239 angle = math.atan2(mouth_vector[1], mouth_vector[0])	angle es la inclinación de la boca respecto a la horizontal.
241 cos_a, sin_a = math.cos(-angle), math.sin(-angle) 242 rotation = np.array([[cos_a, -sin_a], [sin_a, cos_a]], dtype=np.float64)	Arma la matriz que gira todo en sentido contrario a ese ángulo (usa el seno y el coseno de −angle), para dejar la boca horizontal.
244 lip_pts = landmarks_px[LIP_INDICES].astype(np.float64)	Se queda solo con los 20 puntos de los labios (índices 48 a 67) y los pasa a decimales de 64 bits.
245 translated = lip_pts - mouth_center	Traslación: resta el centro de la boca, así el centro queda en (0, 0).
246 rotated = translated @ rotation.T	Rotación: gira todos los puntos con la matriz de arriba.
247 normalized = rotated / (scale + SCALE_EPSILON)	Escala: divide por el ancho de la boca (más una constante diminuta para no dividir por cero).
249 return normalized.astype(np.float32).flatten()	Devuelve los 20 puntos (x, y) como un solo vector de 40 números.

5. normalize_face_landmarks (los 68 puntos, medidos con los ojos)

server/main.py:171-184

Convierte los 68 puntos en píxeles a números que no dependen de qué tan cerca está la cara ni de cuánto está inclinada. Como regla de medir usa los ojos.

`171 def normalize_face_landmarks(landmarks_px):`	Define la función: recibe los puntos en píxeles.
`172 ojo_derecho = landmarks_px[OJO_DERECHO_EXTERNO].astype(np.float64)` `173 ojo_izquierdo = landmarks_px[OJO_IZQUIERDO_EXTERNO].astype(np.float64)`	Toma la esquina externa de cada ojo (puntos 36 y 45).
`174 centro = (ojo_derecho + ojo_izquierdo) / 2.0`	El centro es el punto medio entre los dos ojos.
`175 vector_ojos = ojo_izquierdo - ojo_derecho`	Vector que va de un ojo al otro.
`176 escala = np.linalg.norm(vector_ojos)`	Su largo es la escala: la distancia entre los ojos.
`177 if escala < 1e-3:` `178 return None`	Si los ojos están casi en el mismo punto, la detección es mala y devuelve None.
`179 angulo = math.atan2(vector_ojos[1], vector_ojos[0])`	Ángulo de inclinación de la línea de los ojos.
`180 cos_a, sin_a = math.cos(-angulo), math.sin(-angulo)` `181 rotacion = np.array([[cos_a, -sin_a], [sin_a, cos_a]], dtype=np.float64)`	Arma la matriz que gira los puntos en sentido contrario para enderezar la cara.
`182 puntos = landmarks_px[:68].astype(np.float64)`	Toma los 68 puntos como decimales.
`183 normalizados = ((puntos - centro) @ rotacion.T) / (escala + SCALE_EPSILON)`	Resta el centro, gira, y divide por la escala.
`184 return normalizados.astype(np.float32).flatten()`	Devuelve 136 números (68 puntos x 2 coordenadas).

6. is_outlier (y su constante)

scripts/extraer_landmarks_npy.py:252-258

Detecta cuando el detector de cara se equivocó: si un punto «salta» demasiado de un frame al siguiente, ese frame se descarta.

`252 OUTLIER_MAX_JUMP = 0.15`	Salto máximo permitido entre un frame y el siguiente: 0,15, medido en anchos de boca.
`255 def is_outlier(vector, last_valid_vector):`	Define la función. Recibe el vector del frame actual y el último vector que fue válido.
`256 if last_valid_vector is None:` `257 return False`	Si todavía no hay un frame válido anterior no se puede comparar, así que no se considera atípico.
`258 return float(np.abs(vector - last_valid_vector).max()) > OUTLIER_MAX_JUMP`	Calcula el mayor cambio de cualquier coordenada respecto al último válido. Si pasa de 0,15, es un salto imposible de una boca real: error del detector.

7. fill_missing_frames

scripts/extraer_landmarks_npy.py:261-291

Hay frames donde no se detectó la cara. En vez de borrar el clip, se rellenan esos huecos para no romper la continuidad del movimiento.

`261 def fill_missing_frames(positions):`	Define la función. Recibe una lista con un vector por frame, o None cuando ese frame no tuvo cara.
`262 primeros_validos = [i for i, p in enumerate(positions) if p is not None]`	Lista con las posiciones de los frames que sí tienen dato.
`263 if not primeros_validos:` `264 return None`	Si ninguno tiene dato, devuelve None: ese clip no sirve.
`266 primer_valido = primeros_validos[0]`	Posición del primer frame con dato.

`ultimo_valido_idx = primeros_validos[-1]`	Posición del último frame con dato.
`relleno = list(positions)`	Copia de la lista, para rellenarla sin tocar la original.
`for i in range(primer_valido):` `  relleno[i] = relleno[primer_valido]`	Los frames del principio que no tuvieron cara copian el primer frame válido.
`i = primer_valido`	Empieza a recorrer desde el primer frame válido.
`while i < ultimo_valido_idx:`	Recorre hasta el último frame válido.
`if relleno[i + 1] is None:`	Si el frame siguiente no tiene dato, hay un hueco en medio.

`j = i + 1`
`while relleno[j] is None:`
`  j += 1`
	Busca dónde termina el hueco: avanza j hasta encontrar un frame con dato.

`inicio, fin = relleno[i], relleno[j]`	Toma el dato de cada lado del hueco: el de antes (inicio) y el de después (fin).
`pasos = j - i`	Cuenta cuántos frames hay entre los dos.

`for k in range(1, pasos):`
`  t = k / pasos`
`  relleno[i + k] = (1 - t) * inicio + t * fin`
	Rellena cada frame del hueco con una mezcla lineal entre inicio y fin: t va de 0 a 1 y el valor es (1 − t) · inicio + t · fin. Es una interpolación lineal.

`i = j`	Salta al final del hueco para seguir.
`else:` `  i += 1`	Si el siguiente frame sí tiene dato, solo avanza un frame.
`for i in range(ultimo_valido_idx + 1, len(relleno)):` `  relleno[i] = relleno[ultimo_valido_idx]`	Los frames del final sin cara copian el último frame válido.
`return np.stack(relleno, axis=0).astype(np.float32)`	Apila todo en una matriz (frames × 40) y la devuelve.

8. smooth_positions (y su constante)

scripts/extraer_landmarks_npy.py:294-302

El detector tiembla un poco de un frame a otro. Esto lo suaviza sin perder el movimiento real de la boca.

`SMOOTHING_ALPHA = 0.4`	SMOOTHING_ALPHA = 0,4: cuánto pesa el dato nuevo (40 %) frente al pasado (60 %).
`def smooth_positions(positions, alpha=SMOOTHING_ALPHA):`	Define la función. Recibe la secuencia (frames × 40) y el alpha.
`smoothed = np.empty_like(positions)`	Crea una matriz vacía del mismo tamaño para el resultado.
`smoothed[0] = positions[0]`	El primer frame queda igual.
`for t in range(1, positions.shape[0]):` `  smoothed[t] = alpha * positions[t] + (1 - alpha) * smoothed[t - 1]`	Para cada frame siguiente: 0,4 × dato actual + 0,6 × valor ya suavizado del frame anterior. Es un suavizado exponencial.
`return smoothed.astype(np.float32)`	Devuelve la secuencia suavizada.

9. add_dynamics

scripts/extraer_landmarks_npy.py:305-312

Al modelo no le basta saber dónde están los labios: también necesita cuánto se mueven. Aquí se suman la velocidad y la aceleración, y de 40 números por frame se pasa a 120.

`def add_dynamics(positions):`	Define la función. Recibe la secuencia de posiciones (frames × 40).

`306 velocity = np.zeros_like(positions)`	Crea una matriz de ceros para la velocidad.
`307 velocity[1:] = positions[1:] - positions[:-1]`	Velocidad = posición de este frame − posición del frame anterior. El primer frame se queda con velocidad 0.
`309 acceleration = np.zeros_like(positions)`	Crea una matriz de ceros para la aceleración.
`310 acceleration[1:] = velocity[1:] - velocity[:-1]`	Aceleración = velocidad de este frame − velocidad del anterior.
`312 return np.concatenate([positions, velocity, acceleration], axis=1).astype(np.float32)`	Pega posición, velocidad y aceleración una al lado de la otra: 40 + 40 + 40 = 120 números por frame. Devuelve el resultado.

10. preprocess_sequence (deja la secuencia lista para el modelo)

`server/main.py:536-554`

Es el último paso antes de la red: suaviza, agrega velocidad y aceleración, y deja siempre la misma cantidad de frames (60).

`536 def preprocess_sequence(positions, max_frames):`	Define la función: recibe los puntos de cada frame y el largo fijo (max_frames).
`537 smoothed = smooth_positions(positions)`	Suaviza el movimiento.
`538 sequence_np = add_dynamics(smoothed)`	Agrega velocidad y aceleración (de 136 a 408 números por frame).
`539 sequence = torch.from_numpy(sequence_np).float()`	Lo pasa a un tensor de PyTorch.
`541 T = sequence.shape[0]`	T es cuántos frames tiene el video.

`542 if T < max_frames:`
`543 real_length = T`
`544 padding = torch.zeros((max_frames - T,) + sequence.shape[1:], dtype=sequence.dtype)`
`545 sequence = torch.cat([sequence, padding], dim=0)`
	Si tiene menos de 60, lo rellena con ceros y anota cuántos frames son reales.

`546 elif T > max_frames:`
`547 start = (T - max_frames) // 2`
`548 sequence = sequence[start:start + max_frames]`
`549 real_length = max_frames`
	Si tiene más de 60, se queda con los 60 del medio.

`550 else:` `551 real_length = T`	Si tiene justo 60, no hace nada.
`553 mask = make_padding_mask(torch.tensor([real_length]), max_frames, sequence.device)`	Arma la máscara que le dice al modelo cuáles frames son reales y cuáles son relleno.
`554 return sequence.unsqueeze(0), mask`	Devuelve la secuencia con una dimensión extra de lote (de 1 elemento) y su máscara.

11. process_clip

`scripts/extraer_landmarks_npy.py:315-351`

Es la función que junta todo lo anterior: recibe un video y devuelve la secuencia de 120 números por frame lista para entrenar o predecir.

`315 def process_clip(landmarker, video_path, start_timestamp_ms):`	Define la función: recibe el detector de cara, la ruta del video y un timestamp inicial.
`316 cap = cv2.VideoCapture(video_path)`	Abre el video con OpenCV.
`317 fps = cap.get(cv2.CAP_PROP_FPS) or 25`	Lee los cuadros por segundo del video (25 si no lo sabe).
`318 frame_ms = max(int(1000 / fps), 1)`	Calcula cuántos milisegundos dura cada frame (mínimo 1).

320 `positions = []`	Lista donde se guarda un vector por frame (o None si no hubo cara).
321 `detected, total = 0, 0`	Contadores: cuántos frames tuvieron cara y cuántos hubo en total.
322 `last_valid_vector = None`	Guarda el último vector válido, para comparar saltos.
323 `timestamp_ms = start_timestamp_ms`	Timestamp que va creciendo frame a frame (MediaPipe lo exige).
325 `while True:`	Bucle: recorre el video frame por frame.
326 `ok, frame = cap.read()`	Lee el siguiente frame.
327 `if not ok:` 328 `break`	Si ya no quedan frames, termina el bucle.
329 `total += 1`	Cuenta el frame.
331 `landmarks_px, timestamp_ms = detect_frame_landmarks(landmarker, frame, timestamp_ms, frame_ms)`	Detecta los puntos de la cara en ese frame y devuelve también el siguiente timestamp.
332 `vector = normalize_lip_landmarks(landmarks_px) if landmarks_px is not None else None`	Si hubo cara, normaliza los puntos de los labios; si no, queda en None.
334 `if vector is not None and is_outlier(vector, last_valid_vector):` 335 `vector = None`	Si el vector es un salto imposible respecto al anterior, se descarta (se pone None).
337 `if vector is not None:` 338 `detected += 1` 339 `last_valid_vector = vector`	Si el vector es válido: cuenta ese frame como detectado y lo guarda como el último válido.
340 `positions.append(vector)`	Agrega el vector (o None) a la lista.
342 `cap.release()`	Cierra el video.
343 `next_timestamp_ms = timestamp_ms + frame_ms * 5`	Calcula el timestamp para el siguiente clip, dejando 5 frames de margen.
345 `filled = fill_missing_frames(positions)`	Rellena los huecos con la función de arriba.
346 `if filled is None:` 347 `return None, detected, total, next_timestamp_ms`	Si ningún frame tuvo cara, devuelve None (junto con los conteos y el timestamp): el clip se descarta.
349 `smoothed = smooth_positions(filled)`	Suaviza la secuencia.
350 `sequence = add_dynamics(smoothed)`	Agrega velocidad y aceleración: 120 números por frame.
351 `return sequence, detected, total, next_timestamp_ms`	Devuelve la secuencia lista para el modelo, cuántos frames tuvieron cara, el total y el siguiente timestamp.

Fase 2 · Datos y entrenamiento (PyTorch)

12. LipReadingDataset (cómo se leen los .npy para entrenar)

`scripts/lip_reading_dataset.py:20-96`

Lee los archivos .npy de landmarks, sabe de qué palabra es cada uno por el nombre de su carpeta, y se los entrega al entrenamiento ya con el mismo largo.

`20 class LipReadingDataset(Dataset):`	Define la clase; hereda de Dataset de PyTorch.
`22 def __init__(self, data_dir=DEFAULT_DATA_DIR, max_frames=60, file_paths=None, augment=False):`	El constructor recibe la carpeta de datos, el largo fijo de frames, la lista de archivos y si hay aumento de datos.

`23 self.data_dir = data_dir`
`24 self.max_frames = max_frames`
`25 self.augment = augment`
	Guarda esos datos.

`27 label_map_path = os.path.join(data_dir, "label_map.json")`
`28 if not os.path.exists(label_map_path):`
`29 raise FileNotFoundError(`
`30 f"No encontré {label_map_path}. Corré extraer_landmarks_npy.py primero."`
`31 )`
	Busca label_map.json (el número de cada clase); si no existe, avisa que hay que extraer los landmarks primero.

`32 with open(label_map_path, "r", encoding="utf-8") as f:` `33 raw_label_map = json.load(f)`	Lee ese archivo.
`34 self.label_map = {_nfc(word): idx for word, idx in raw_label_map.items()}`	Normaliza los nombres de las clases para que las tildes siempre se escriban igual.
`36 num_classes = max(self.label_map.values()) + 1`	Cuenta cuántas clases hay.

`37 self.class_names = ["?"] * num_classes`
`38 for word, idx in self.label_map.items():`
`39 self.class_names[idx] = word`
	Arma la lista con el nombre de cada clase en su posición.

`41 if file_paths is not None:`
`42 self.file_paths = list(file_paths)`
`43 else:`
`44 self.file_paths = sorted(glob.glob(os.path.join(data_dir, "*", "*.npy")))`
	Usa la lista de archivos que le dieron o, si no, busca todos los .npy de la carpeta.

`45 if not self.file_paths:` `46 raise RuntimeError(f"No hay archivos .npy en {data_dir}.")`	Si no hay ningún .npy, falla con un aviso.
`48 first_sample = np.load(self.file_paths[0])` `49 self.landmark_dim = int(first_sample.shape[-1])`	Abre el primer archivo para saber cuántos números tiene cada frame: 120 con labios, 408 con la cara completa.
`51 def __len__(self):` `52 return len(self.file_paths)`	Cuántos clips hay.

`54 def _label_from_path(self, path):`
`55 word = _nfc(os.path.basename(os.path.dirname(path)))`
`56 return self.label_map[word]`
	La clase de un clip es el nombre de su carpeta, convertido a número con label_map.

`58 def _apply_augmentation(self, sequence):`	Aumento de datos: pequeñas alteraciones al azar para que el modelo no memorice.
`59 sequence = sequence + np.random.normal(0, 0.003, sequence.shape).astype(np.float32)`	Le suma un poquito de ruido.

`61 shift = np.random.uniform(-0.01, 0.01, size=(1, sequence.shape[1])).astype(np.float32)`
`62 scale = np.random.uniform(0.97, 1.03)`
`63 sequence = sequence * scale + shift`
	Lo desplaza y lo agranda o achica un poquito.

65 T = sequence.shape[0] 66 n_mask = max(1, int(round(T * np.random.uniform(0.08, 0.12)))) 67 n_mask = min(n_mask, T) 68 mask_idx = np.random.choice(T, size=n_mask, replace=False) 69 sequence[mask_idx] = 0.0	Apaga entre 8 % y 12 % de los frames al azar, como si el detector hubiera fallado.
71 return sequence.astype(np.float32)	Devuelve la secuencia alterada.
73 def __getitem__(self, idx):	__getitem__: entrega el clip número idx.
74 path = self.file_paths[idx] 75 sequence = np.load(path).astype(np.float32) 76 label = self._label_from_path(path)	Carga el .npy y calcula su clase.
78 if self.augment: 79 sequence = self._apply_augmentation(sequence)	Si es entrenamiento, aplica el aumento de datos.
81 T = sequence.shape[0] 82 if T < self.max_frames: 83 pad = np.zeros((self.max_frames - T, sequence.shape[1]), dtype=np.float32) 84 sequence = np.concatenate([sequence, pad], axis=0) 85 real_length = T 86 elif T > self.max_frames: 87 sequence = sequence[: self.max_frames] 88 real_length = self.max_frames 89 else: 90 real_length = T	Deja el clip con exactamente max_frames: rellena con ceros si es corto, corta si es largo, y anota cuántos frames son reales.
92 return (93 torch.from_numpy(sequence), 94 torch.tensor(label, dtype=torch.long), 95 torch.tensor(real_length, dtype=torch.long), 96)	Devuelve la secuencia, la clase y el largo real.

13. build_dataloaders (separar entrenamiento y validación)

scripts/lip_reading_dataset.py:99-130

Reparte los clips: 80 % para entrenar y 20 % para validar, manteniendo la misma proporción en cada clase. La semilla fija hace que la separación sea siempre la misma.

99 def build_dataloaders(data_dir=DEFAULT_DATA_DIR, max_frames=60, batch_size=32, 100 val_ratio=0.2, seed=42, num_workers=0, augment_train=True):	Define la función con sus valores por defecto: 60 frames, lotes de 32, 20 % de validación y semilla 42.
101 reference = LipReadingDataset(data_dir=data_dir, max_frames=max_frames) 102 file_paths = reference.file_paths	Crea un dataset con todos los clips solo para conocer la lista de archivos.
104 labels_by_path = {p: reference.label_map[_nfc(os.path.basename(os.path.dirname(p)))] for p in file_paths}	Calcula la clase de cada archivo.
105 paths_by_class = {} 106 for p in file_paths: 107 paths_by_class.setdefault(labels_by_path[p], []).append(p)	Agrupa los archivos por clase.
109 rng = np.random.default_rng(seed) 110 train_paths, val_paths = [], []	Generador de azar con semilla fija y las dos listas vacías.
111 for label, paths in paths_by_class.items(): 112 paths = list(paths) 113 rng.shuffle(paths) 114 n_val = max(1, int(round(len(paths) * val_ratio))) 115 val_paths.extend(paths[:n_val]) 116 train_paths.extend(paths[n_val:])	Por cada clase: mezcla sus archivos, separa el 20 % (al menos 1) para validar y el resto para entrenar.
117 rng.shuffle(train_paths) 118 rng.shuffle(val_paths)	Mezcla otra vez las dos listas.

`120 train_ds = LipReadingDataset(data_dir=data_dir, max_frames=max_frames,` `121 file_paths=train_paths, augment=augment_train)`	Dataset de entrenamiento, con aumento de datos.
`122 val_ds = LipReadingDataset(data_dir=data_dir, max_frames=max_frames,` `123 file_paths=val_paths, augment=False)`	Dataset de validación, sin aumento.
`125 train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,` `126 num_workers=num_workers, drop_last=False)`	Cargador de entrenamiento: entrega lotes de 32 y los mezcla cada vuelta.
`127 val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,` `128 num_workers=num_workers, drop_last=False)`	Cargador de validación: sin mezclar.
`130 return reference, train_loader, val_loader`	Devuelve el dataset de referencia y los dos cargadores.

14. PositionalEncoding

`scripts/train_landmarks_transformer.py:29-40`

El Conformer mira todos los frames a la vez y no sabe cuál va primero. Esto le suma a cada frame una «marca de posición» para que sepa el orden.

`29 class PositionalEncoding(nn.Module):`	Define la clase como una capa de PyTorch (nn.Module).
`30 def __init__(self, d_model, max_len=500):`	Constructor: recibe el tamaño del vector (d_model) y la longitud máxima (500 frames).
`31 super().__init__()`	Inicializa la clase base de PyTorch.
`32 pe = torch.zeros(max_len, d_model)`	Matriz de ceros (500 × d_model) donde irán las marcas.
`33 position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)`	Vector con las posiciones 0, 1, 2… en forma de columna.
`34 div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))`	Calcula las frecuencias que se usan para las ondas (más lentas o más rápidas según la dimensión).
`35 pe[:, 0::2] = torch.sin(position * div_term)`	En las columnas pares escribe un seno de la posición.
`36 pe[:, 1::2] = torch.cos(position * div_term)`	En las columnas impares escribe un coseno de la posición.
`37 self.register_buffer("pe", pe.unsqueeze(0))`	Guarda la matriz como un «buffer»: viaja con el modelo pero no se entrena.
`39 def forward(self, x):`	forward: lo que hace la capa al recibir datos x.
`40 return x + self.pe[:, : x.size(1)]`	Le suma a x las marcas de posición de los frames que tiene.

15. TCNBlock

`scripts/train_landmarks_transformer.py:43-62`

Un bloque de la red convolucional temporal (TCN). Mira unos pocos frames vecinos a la vez para captar movimientos rápidos de la boca, y suma su entrada a su salida para no perder información.

`43 class TCNBlock(nn.Module):`	Define la clase como capa de PyTorch.
`44 def __init__(self, channels, kernel_size=3, dilation=1, dropout=0.1):`	Constructor: canales, tamaño de la ventana (kernel 3), dilatación y dropout.

`45 super().__init__()`	Inicializa la clase base.
`46 padding = (kernel_size - 1) * dilation // 2`	Calcula el relleno necesario para que la salida tenga la misma longitud que la entrada.
`47 self.conv1 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)`	Primera convolución en una dimensión (en el tiempo), con la dilatación indicada.
`48 self.bn1 = nn.BatchNorm1d(channels)`	Normalización por lotes: estabiliza los números.
`49 self.conv2 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)`	Segunda convolución igual.
`50 self.bn2 = nn.BatchNorm1d(channels)`	Segunda normalización.
`51 self.relu = nn.ReLU()`	Función de activación ReLU: deja pasar los positivos y pone 0 a los negativos.
`52 self.dropout = nn.Dropout(dropout)`	Dropout: apaga al azar algunas neuronas al entrenar para que no memorice.
`54 def forward(self, x, key_padding_mask=None):`	forward: lo que hace el bloque con los datos x y la máscara de relleno.
`55 residual = x`	Guarda la entrada original (conexión residual).
`56 x = x.transpose(1, 2)`	Intercambia ejes para que el tiempo quede donde la convolución lo espera.
`57 if key_padding_mask is not None:` `58 x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)`	Si hay máscara, pone ceros en los frames de relleno para que no aporten nada.
`59 x = self.dropout(self.relu(self.bn1(self.conv1(x))))`	Primera convolución, normalización, ReLU y dropout.
`60 x = self.bn2(self.conv2(x))`	Segunda convolución y normalización.
`61 x = x.transpose(1, 2)`	Devuelve los ejes a su lugar.
`62 return self.relu(residual + self.dropout(x))`	Suma la entrada original (residual) a la salida, aplica ReLU y devuelve el resultado.

16. TCNFeatureExtractor

scripts/train_landmarks_transformer.py:65-76

Apila varios TCNBlock. Cada uno mira una ventana más ancha que el anterior (dilatación 1, 2, 4), así la red capta movimientos cortos y largos.

`65 class TCNFeatureExtractor(nn.Module):`	Define la clase como capa de PyTorch.
`66 def __init__(self, channels, n_blocks=3, kernel_size=3, dropout=0.1):`	Constructor: canales, cantidad de bloques (3), tamaño de ventana y dropout.
`67 super().__init__()`	Inicializa la clase base.

`68 self.blocks = nn.ModuleList([`
`69 TCNBlock(channels, kernel_size=kernel_size, dilation=2 ** i, dropout=dropout)`
`70 for i in range(n_blocks)`
`71 ])`
	Crea la lista de bloques. El bloque i usa dilatación 2 elevado a i: 1, 2 y 4.

`73 def forward(self, x, key_padding_mask=None):`	forward: lo que hace con los datos x.
`74 for block in self.blocks:` `75 x = block(x, key_padding_mask=key_padding_mask)`	Pasa los datos por cada bloque, uno tras otro.
`76 return x`	Devuelve el resultado.

17. ConformerFeedForward

scripts/train_landmarks_transformer.py:79-93

Una mini red de dos capas lineales que procesa cada frame por separado. Es una de las piezas dentro de cada bloque Conformer.

`class ConformerFeedForward(nn.Module):`	Define la clase como capa de PyTorch.
`def __init__(self, d_model, dim_feedforward, dropout):`	Constructor: tamaño del vector, tamaño interno y dropout.
`super().__init__()`	Inicializa la clase base.

`self.net = nn.Sequential(`
`nn.LayerNorm(d_model),`
`nn.Linear(d_model, dim_feedforward),`
`nn.SiLU(),`
`nn.Dropout(dropout),`
`nn.Linear(dim_feedforward, d_model),`
`nn.Dropout(dropout),`
`)`
	Arma una secuencia de capas: normaliza, sube el tamaño con una capa lineal, aplica la activación SiLU, dropout, baja al tamaño original con otra capa lineal y otro dropout.

`def forward(self, x):`	forward: lo que hace con los datos x.
`return self.net(x)`	Pasa x por esa secuencia y devuelve el resultado.

18. ConformerConvModule

`scripts/train_landmarks_transformer.py:96-128`

La parte «convolucional» del Conformer: capta relaciones locales entre frames cercanos (como el TCN), pero dentro de cada bloque Conformer.

`class ConformerConvModule(nn.Module):`	Define la clase como capa de PyTorch.
`def __init__(self, d_model, kernel_size=15, dropout=0.1):`	Constructor: tamaño del vector, ventana de la convolución (15) y dropout.
`super().__init__()`	Inicializa la clase base.
`self.layer_norm = nn.LayerNorm(d_model)`	Normalización de capa.
`self.pointwise_conv1 = nn.Conv1d(d_model, 2 * d_model, kernel_size=1)`	Convolución de tamaño 1 que duplica los canales.
`self.glu = nn.GLU(dim=1)`	GLU: una compuerta que usa la mitad de los canales para decidir cuánto deja pasar de la otra mitad.
`padding = (kernel_size - 1) // 2`	Relleno para que la salida mida lo mismo que la entrada.

`self.depthwise_conv = nn.Conv1d(`
`d_model, d_model, kernel_size=kernel_size, padding=padding, groups=d_model`
`)`
	Convolución «depthwise»: cada canal se procesa por separado con una ventana de 15 frames.

`self.batch_norm = nn.BatchNorm1d(d_model)`	Normalización por lotes.
`self.swish = nn.SiLU()`	Activación SiLU (también llamada swish).
`self.pointwise_conv2 = nn.Conv1d(d_model, d_model, kernel_size=1)`	Convolución de tamaño 1 que vuelve al tamaño original.
`self.dropout = nn.Dropout(dropout)`	Dropout.
`def forward(self, x, key_padding_mask=None):`	forward: lo que hace con los datos x y la máscara.
`residual = x`	Guarda la entrada (conexión residual).
`x = self.layer_norm(x)`	Normaliza.
`x = x.transpose(1, 2)`	Intercambia ejes para que el tiempo quede donde la convolución lo espera.

117 if key_padding_mask is not None: 118 x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)	Si hay máscara, pone ceros en los frames de relleno.
120 x = self.pointwise_conv1(x) 121 x = self.glu(x)	Convolución de tamaño 1 y compuerta GLU.
122 x = self.depthwise_conv(x)	Convolución depthwise de 15 frames.
123 x = self.batch_norm(x) 124 x = self.swish(x)	Normalización y activación SiLU.
125 x = self.pointwise_conv2(x)	Convolución de tamaño 1 final.
126 x = self.dropout(x)	Dropout.
127 x = x.transpose(1, 2)	Devuelve los ejes a su lugar.
128 return residual + x	Suma la entrada original y devuelve el resultado.

19. ConformerBlock

scripts/train_landmarks_transformer.py:131-161

Un bloque completo del Conformer: combina atención (mira todos los frames a la vez y decide cuáles importan) con convolución (relaciones locales). Es la parte que entiende la secuencia entera del movimiento.

131 class ConformerBlock(nn.Module):	Define la clase como capa de PyTorch.
132 def __init__(self, d_model, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):	Constructor: tamaño del vector, cabezas de atención, tamaño interno, ventana de convolución y dropout.
133 super().__init__()	Inicializa la clase base.
134 self.ff1 = ConformerFeedForward(d_model, dim_feedforward, dropout)	Primera mini red feed-forward.
136 self.self_attn_norm = nn.LayerNorm(d_model)	Normalización antes de la atención.
137 self.self_attn = nn.MultiheadAttention(138 d_model, n_heads, dropout=dropout, batch_first=True 139)	Capa de atención con varias cabezas: cada cabeza mira la secuencia desde un ángulo distinto.
140 self.attn_dropout = nn.Dropout(dropout)	Dropout para la salida de la atención.
142 self.conv_module = ConformerConvModule(d_model, conv_kernel_size, dropout)	Módulo convolucional.
144 self.ff2 = ConformerFeedForward(d_model, dim_feedforward, dropout)	Segunda mini red feed-forward.
145 self.final_norm = nn.LayerNorm(d_model)	Normalización final del bloque.
147 def forward(self, x, src_key_padding_mask=None):	forward: lo que hace con los datos x y la máscara.
148 x = x + 0.5 * self.ff1(x)	Suma la mitad de la salida del primer feed-forward.
150 residual = x	Guarda x para sumarlo después (residual).
151 x_norm = self.self_attn_norm(x)	Normaliza antes de la atención.
152 attn_out, _ = self.self_attn(153 x_norm, x_norm, x_norm, key_padding_mask=src_key_padding_mask, need_weights=False 154)	Autoatención: cada frame compara con todos los demás (consulta, clave y valor son lo mismo); la máscara ignora los frames de relleno.
155 x = residual + self.attn_dropout(attn_out)	Suma la salida de la atención (con dropout) a lo guardado.
157 x = self.conv_module(x, key_padding_mask=src_key_padding_mask)	Pasa por el módulo convolucional.
159 x = x + 0.5 * self.ff2(x)	Suma la mitad de la salida del segundo feed-forward.
161 return self.final_norm(x)	Normaliza y devuelve el resultado del bloque.

20. ConformerEncoder

scripts/train_landmarks_transformer.py:164-176

Apila los bloques Conformer (en este proyecto, 2 capas).

`class ConformerEncoder(nn.Module):`	Define la clase como capa de PyTorch.
`def __init__(self, d_model, n_layers, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):`	Constructor: tamaño del vector, cantidad de capas, cabezas, tamaño interno, ventana y dropout.
`super().__init__()`	Inicializa la clase base.
`self.layers = nn.ModuleList([ ConformerBlock(d_model, n_heads, dim_feedforward, conv_kernel_size, dropout) for _ in range(n_layers) ])`	Crea la lista de bloques Conformer, tantos como n_layers.
`def forward(self, x, src_key_padding_mask=None):`	forward: lo que hace con los datos x y la máscara.
`for layer in self.layers: x = layer(x, src_key_padding_mask=src_key_padding_mask)`	Pasa x por cada bloque, uno tras otro.
`return x`	Devuelve el resultado.

21. LipReadingConformer (el modelo de video)

scripts/train_landmarks_transformer.py:179-214

Es tu modelo de video completo. Recibe la secuencia de landmarks (60 frames × 120 números) y devuelve una probabilidad por cada clase (por ejemplo carro_correcto o carro_incorrecto).

`class LipReadingConformer(nn.Module):`	Define la clase como capa de PyTorch.
`def __init__(self, num_classes, input_dim, hidden_dim=128, n_layers=2, n_heads=8, dim_feedforward=256, conv_kernel_size=15, tcn_blocks=3, tcn_kernel_size=3, dropout=0.3):`	Constructor. Recibe el número de clases y el tamaño de entrada (120 con labios, 408 con cara completa). Los valores por defecto son los del proyecto: tamaño interno 128, 2 capas Conformer, 8 cabezas de atención, ventana 15, 3 bloques TCN, dropout 0,3.
`super().__init__()`	Inicializa la clase base.
`self.input_proj = nn.Linear(input_dim, hidden_dim)`	Capa lineal que lleva los números de entrada (120 o 408) al tamaño interno de 128.
`self.tcn = TCNFeatureExtractor( hidden_dim, n_blocks=tcn_blocks, kernel_size=tcn_kernel_size, dropout=dropout )`	El extractor TCN: capta los movimientos rápidos entre frames vecinos.
`self.pos_encoder = PositionalEncoding(hidden_dim)`	La marca de posición para que el Conformer sepa el orden de los frames.
`self.conformer_encoder = ConformerEncoder( d_model=hidden_dim, n_layers=n_layers, n_heads=n_heads, dim_feedforward=dim_feedforward, conv_kernel_size=conv_kernel_size, dropout=dropout, )`	El codificador Conformer: entiende la secuencia completa.
`self.classifier = nn.Sequential( nn.Linear(hidden_dim, 64), nn.ReLU(), nn.Dropout(dropout), nn.Linear(64, num_classes), )`	El clasificador final: baja de 128 a 64, aplica ReLU y dropout, y termina con una capa que da un puntaje por clase.
`def forward_features(self, x, src_key_padding_mask=None):`	forward_features: resume la secuencia en un solo vector.
`x = self.input_proj(x)`	Lleva la entrada al tamaño interno.

`207 x = self.tcn(x, key_padding_mask=src_key_padding_mask)`	Pasa por el TCN.
`208 x = self.pos_encoder(x)`	Suma las marcas de posición.
`209 x = self.conformer_encoder(x, src_key_padding_mask=src_key_padding_mask)`	Pasa por el codificador Conformer.
`210 return masked_mean_pool(x, src_key_padding_mask)`	Promedia los frames reales (sin contar el relleno) y devuelve un vector.
`212 def forward(self, x, src_key_padding_mask=None):`	forward: lo que hace el modelo al recibir una secuencia.
`213 pooled = self.forward_features(x, src_key_padding_mask=src_key_padding_mask)`	Resume la secuencia en un vector.
`214 return self.classifier(pooled)`	Lo pasa por el clasificador y devuelve los puntajes de cada clase.

22. masked_mean_pool y make_padding_mask

scripts/train_landmarks_transformer.py:217-229

Los clips tienen distinta duración, así que se rellenan con ceros hasta 60 frames. Estas dos funciones evitan que el relleno cuente.

`217 def masked_mean_pool(x, src_key_padding_mask):`	masked_mean_pool: recibe los datos x y la máscara de relleno.
`218 if src_key_padding_mask is None:` `219 return x.mean(dim=1)`	Si no hay máscara, promedia todos los frames.
`221 real_mask = (~src_key_padding_mask).unsqueeze(-1).float()`	Invierte la máscara: 1 en los frames reales y 0 en el relleno.
`222 summed = (x * real_mask).sum(dim=1)`	Suma los frames reales.
`223 counts = real_mask.sum(dim=1).clamp(min=1.0)`	Cuenta cuántos frames reales hay (mínimo 1, para no dividir por cero).
`224 return summed / counts`	Devuelve la suma dividida entre la cuenta: el promedio de los frames reales.
`227 def make_padding_mask(lengths, max_len, device):`	make_padding_mask: recibe la duración real de cada clip, el largo máximo y el dispositivo.
`228 idx = torch.arange(max_len, device=device).unsqueeze(0)`	Crea los números 0, 1, 2… hasta el largo máximo.
`229 return idx >= lengths.unsqueeze(1)`	Marca como relleno (True) los frames cuya posición es mayor o igual a la duración real.

23. run_epoch (una vuelta por todos los datos)

scripts/train_landmarks_transformer.py:272-302

Una «época» es una pasada completa por todos los clips. Esta función la hace, tanto para entrenar (corrigiendo al modelo) como para validar (solo medir cómo lo hace con datos que no vio).

`272 def run_epoch(model, loader, criterion, optimizer, device, train):`	Define la función: recibe el modelo, los datos, la función de pérdida, el optimizador, el dispositivo y entrenar (True para entrenar, False para validar).
`273 model.train() if train else model.eval()`	Pone el modelo en modo entrenamiento o en modo evaluación.
`274 total_loss, correct, total = 0.0, 0, 0`	Contadores: pérdida acumulada, aciertos y total de clips.
`276 context = torch.enable_grad() if train else torch.no_grad()`	Si entrena, calcula gradientes; si valida, no los calcula (más rápido y sin modificar el modelo).

277 `with context:`	Abre ese contexto.
278 `for sequences, labels, lengths in loader:`	Recorre los datos en lotes: secuencias, etiquetas y duraciones reales.
279 `sequences = sequences.to(device)` 280 `labels = labels.to(device)` 281 `lengths = lengths.to(device)`	Pasa los tres a la GPU (o CPU).
282 `mask = make_padding_mask(lengths, sequences.shape[1], device)`	Crea la máscara que marca cuáles frames son relleno.
284 `if train:` 285 `optimizer.zero_grad()`	Si entrena, borra los gradientes del lote anterior.
287 `outputs = model(sequences, src_key_padding_mask=mask)`	Pasa las secuencias por el modelo y obtiene los puntajes de cada clase.
288 `loss = criterion(outputs, labels)`	Calcula la pérdida: qué tan lejos están esos puntajes de las etiquetas reales.
290 `if train:` 291 `loss.backward()` 292 `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` 293 `optimizer.step()`	Si entrena: calcula los gradientes (backward), los limita a una norma de 1,0 para que no exploten y actualiza los pesos (step).
295 `total_loss += loss.item() * sequences.size(0)`	Acumula la pérdida del lote.
296 `_, predicted = outputs.max(1)`	La clase predicha es la de mayor puntaje.
297 `correct += predicted.eq(labels).sum().item()`	Cuenta los aciertos.
298 `total += labels.size(0)`	Cuenta los clips procesados.
300 `avg_loss = total_loss / total if total > 0 else 0.0`	Pérdida promedio de toda la época.
301 `acc = 100.0 * correct / total if total > 0 else 0.0`	Porcentaje de acierto de la época.
302 `return avg_loss, acc`	Devuelve la pérdida y el acierto.

24. train (el modelo de video)

`scripts/train_landmarks_transformer.py:407-489`

Es el programa de entrenamiento del modelo de video: carga los landmarks, crea el modelo, entrena época por época y guarda el mejor. Hiperparámetros por defecto: 50 épocas, lotes de 32, 60 frames, tasa de aprendizaje 0,001, 20 % para validar.

407 `def train(epochs=50, batch_size=32, max_frames=60, lr=1e-3, weight_decay=1e-3,` 408 `val_ratio=0.2, early_stopping_patience=15, warmup_epochs=5,` 409 `label_smoothing=0.1):`	Define la función con sus valores por defecto: épocas, tamaño de lote, frames por clip, tasa de aprendizaje, decaimiento de pesos, porcentaje de validación, paciencia de la parada temprana, épocas de calentamiento y suavizado de etiquetas.
410 `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")` 411 `print(f"Entrenando en: {device}")`	Usa la GPU si hay; si no, la CPU, y lo avisa.
413 `dataset, train_loader, val_loader = build_dataloaders(` 414 `data_dir=RUTA_DATASET, max_frames=max_frames, batch_size=batch_size,` 415 `val_ratio=val_ratio, seed=SEED, augment_train=True,` 416 `)`	Crea los cargadores de datos: separa los clips en entrenamiento (80 %) y validación (20 %), con la misma semilla para que el reparto sea repetible.
417 `num_classes = len(dataset.class_names)` 418 `input_dim = dataset.landmark_dim`	Cuenta las clases y detecta el tamaño de entrada (120 con labios, 408 con cara completa) mirando los archivos.
419 `print(f"Clases ({num_classes}): {dataset.class_names}")` 420 `print(f"Dimensión de entrada detectada: {input_dim}")` 421 `print(f"Total muestras: {len(dataset)} | Train: {len(train_loader.dataset)} | "` 422 `f"Val: {len(val_loader.dataset)}")`	Imprime las clases, la dimensión de entrada y cuántos clips hay para entrenar y validar.

424 model = LipReadingConformer(num_classes=num_classes, input_dim=input_dim).to(device)	Crea el modelo LipReadingConformer con esas clases y esa entrada, y lo pasa a la GPU.
426 train_labels = [dataset._label_from_path(p) for p in train_loader.dataset.file_paths]	Saca las etiquetas de los clips de entrenamiento.
427 class_counts = np.bincount(train_labels, minlength=num_classes).astype(np.float64)	Cuenta cuántos clips tiene cada clase.
428 class_counts[class_counts == 0] = 1.0	Si alguna clase tiene 0, le pone 1 para no dividir por cero.
429 class_weights = class_counts.sum() / (num_classes * class_counts)	Calcula un peso por clase: las clases con menos clips pesan más, para compensar.
430 class_weights_t = torch.tensor(class_weights, dtype=torch.float32, device=device)	Pasa los pesos a un tensor.
431 print("Pesos por clase (loss):") 432 for name, w, c in zip(dataset.class_names, class_weights, class_counts): 433 print(f" {name:28s} {int(c):4d} muestras -- peso {w:.3f}")	Imprime cuántas muestras y qué peso tiene cada clase.
435 criterion = nn.CrossEntropyLoss(label_smoothing=label_smoothing, weight=class_weights_t)	Función de pérdida: entropía cruzada con los pesos por clase y suavizado de etiquetas (0,1), que evita que el modelo se crea demasiado seguro.
436 optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)	Optimizador AdamW: decide cuánto corregir cada peso.
437 scheduler = build_warmup_cosine_scheduler(optimizer, epochs, warmup_epochs)	Planificador: sube la tasa de aprendizaje durante las primeras 5 épocas y luego la baja suavemente (forma de coseno).
438 early_stopping = EarlyStopping(patience=early_stopping_patience, min_delta=0.001)	Parada temprana: si la pérdida de validación no mejora durante 15 épocas seguidas, se detiene.
440 os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)	Crea la carpeta donde se guardará el modelo.
441 best_val_loss = float("inf")	La mejor pérdida de validación hasta ahora es infinito.
443 history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}	Diccionario donde se guarda la historia de pérdidas y aciertos, para graficarla.
445 print("\nIniciando entrenamiento (Conformer)...")	Avisa que empieza.
446 for epoch in range(epochs):	Bucle principal: una vuelta por época.
447 train_loss, train_acc = run_epoch(model, train_loader, criterion, optimizer, device, train=True)	Entrena una época con los datos de entrenamiento.
448 val_loss, val_acc = run_epoch(model, val_loader, criterion, optimizer, device, train=False)	Valida con los datos de validación, sin corregir el modelo.
449 scheduler.step()	Avanza el planificador de la tasa de aprendizaje.
451 history["train_loss"].append(train_loss) 452 history["val_loss"].append(val_loss) 453 history["train_acc"].append(train_acc) 454 history["val_acc"].append(val_acc)	Anota los cuatro números de esta época en la historia.
456 lr_actual = optimizer.param_groups[0]["lr"]	Lee la tasa de aprendizaje actual.
457 print(f"Época [{epoch+1}/{epochs}] | Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | " 458 f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}% | LR: {lr_actual:.6f}")	Imprime la época con pérdida y acierto de entrenamiento y validación.
460 if val_loss < best_val_loss: 461 best_val_loss = val_loss	Si esta época tiene la mejor pérdida de validación hasta ahora, la guarda como la mejor.

462 torch.save({ 463 "model_state_dict": model.state_dict(), 464 "class_names": dataset.class_names, 465 "max_frames": max_frames, 466 "input_dim": input_dim, 467 "val_loss": val_loss, 468 "val_acc": val_acc, 469 }, MODEL_PATH)	Guarda en models/best.pth los pesos del modelo, los nombres de las clases, los frames por clip, el tamaño de entrada y las métricas.
470 print(f" -> Nuevo mejor modelo guardado (Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%)")	Avisa que guardó un modelo mejor.
472 early_stopping(val_loss)	Informa la pérdida de validación a la parada temprana.
473 if early_stopping.early_stop: 474 print(f" -> Early stopping activado en la época {epoch+1} " 475 f"(sin mejora en Val Loss por {early_stopping_patience} épocas).") 476 break	Si la parada temprana decide detenerse, lo avisa y sale del bucle.
478 plot_training_curves(history["train_loss"], history["val_loss"], 479 history["train_acc"], history["val_acc"])	Grafica las curvas de pérdida y acierto.
481 print(f"\nListo. Mejor Val Loss: {best_val_loss:.4f} | Modelo en: {MODEL_PATH}")	Avisa que terminó y dónde quedó el modelo.
483 try:	Intenta generar el reporte final.
484 best_checkpoint = torch.load(MODEL_PATH, map_location=device)	Carga el mejor modelo guardado.
485 model.load_state_dict(best_checkpoint["model_state_dict"])	Se lo pone al modelo.
486 evaluate_final_report(model, val_loader, dataset.class_names, device)	Calcula el reporte de métricas (Top-1, Top-3, acierto binario) sobre la validación.
487 except Exception as e: 488 print(f"\n[AVISO] No se pudo generar el reporte de resultados: {e}") 489 print("El modelo entrenado SÍ se guardó bien, esto solo afecta al reporte de métricas.")	Si algo falla al hacer el reporte, lo avisa, pero aclara que el modelo sí se guardó.

25. load_model (carga wav2vec2)

scripts/audio_pronunciation.py:129-152

Carga el modelo de voz wav2vec2 que convierte el audio en fonemas. Es un modelo ya entrenado por otros (no lo entrenaste tú).

129 def load_model(device=None):	Define la función; el dispositivo es opcional.
130 import json 131 from huggingface_hub import hf_hub_download 132 from transformers import Wav2Vec2Config, Wav2Vec2FeatureExtractor, Wav2Vec2ForCTC	Importa lo necesario: lector de json, descargador de Hugging Face y las clases de wav2vec2.
134 device = device or ("cuda" if torch.cuda.is_available() else "cpu") 135 print(f"Cargando {MODEL_NAME} (primera vez tarda, baja ~1.2GB)...")	Elige la GPU si hay, y avisa que la primera vez descarga unos 1,2 GB.
137 feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(MODEL_NAME)	Carga el preparador de audio: lo deja en el formato que el modelo espera.
139 config = Wav2Vec2Config.from_pretrained(MODEL_NAME) 140 model = Wav2Vec2ForCTC(config)	Lee la configuración del modelo y crea el modelo vacío con ella.
141 weights_path = hf_hub_download(MODEL_NAME, "pytorch_model.bin") 142 state_dict = torch.load(weights_path, map_location="cpu", weights_only=True) 143 model.load_state_dict(state_dict)	Descarga los pesos ya entrenados y los carga en el modelo.

144 model.to(device) 145 model.eval()	Lo manda al dispositivo y lo pone en modo evaluación (sin entrenar).
147 vocab_path = hf_hub_download(MODEL_NAME, "vocab.json") 148 with open(vocab_path, encoding="utf-8") as f: 149 vocab = json.load(f)	Descarga y lee el vocabulario: qué número le toca a cada fonema.
151 print(f"Listo. Device: {device}. Vocabulario: {len(vocab)} fonemas.") 152 return model, feature_extractor, vocab, device	Avisa que está listo y devuelve el modelo, el preparador, el vocabulario y el dispositivo.

26. load_audio_16k

scripts/audio_pronunciation.py:155-161

Abre un archivo de audio y lo deja en el formato que espera wav2vec2: un solo canal y 16.000 muestras por segundo.

155 def load_audio_16k(wav_path):	Define la función: recibe la ruta del .wav.
156 waveform, sr = torchaudio.load(wav_path)	Carga el audio: devuelve la onda (waveform) y su frecuencia de muestreo (sr).
157 if waveform.shape[0] > 1: 158 waveform = waveform.mean(dim=0, keepdim=True)	Si el audio tiene más de un canal (estéreo), saca el promedio de los canales para dejar uno solo (mono).
159 if sr != TARGET_SAMPLE_RATE: 160 waveform = torchaudio.functional.resample(waveform, sr, TARGET_SAMPLE_RATE)	Si la frecuencia no es 16.000 Hz, lo remuestrea a 16.000 Hz.
161 return waveform.squeeze(0)	Quita la dimensión sobrante y devuelve la onda como un vector de una dimensión.

27. extract_embedding (y su constante)

scripts/audio_pronunciation.py:164-173

Convierte un audio en un vector de 1.024 números que resume cómo suena. Ese vector es lo que recibe el clasificador de audio (AudioClassifierHead).

164 EMBEDDING_DIM = 1024	Tamaño del vector que resume el audio: 1.024 números.
167 def extract_embedding(wav_path, model, feature_extractor, device):	Define la función: recibe la ruta del audio, el modelo wav2vec2, su preprocesador y el dispositivo (CPU o GPU).
168 waveform = load_audio_16k(wav_path)	Carga el audio ya en 16 kHz y mono.
169 inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")	El preprocesador de wav2vec2 normaliza la onda y la pone en el formato que el modelo pide.
170 with torch.inference_mode():	Modo de inferencia: no calcula gradientes, porque solo se está usando el modelo, no entrenándolo.
171 hidden = model.wav2vec2(inputs.input_values.to(device)).last_hidden_state	model.wav2vec2 es la parte base del modelo (sin la cabeza que reconoce fonemas). Devuelve un vector de 1.024 números por cada instante del audio.
172 pooled = hidden.mean(dim=1).squeeze(0)	Promedia esos vectores en el tiempo: queda un solo vector de 1.024 números para todo el clip.
173 return pooled.cpu().numpy()	Lo pasa a la CPU y lo devuelve como arreglo de NumPy.

28. AudioClassifierHead (el modelo de audio)

scripts/train_audio_classifier.py:95-107

Es tu modelo de audio. Es una red muy chica: recibe el vector de 1.024 números del audio (extract_embedding) y decide a qué clase pertenece.

95 `class AudioClassifierHead(nn.Module):`	Define la clase como capa de PyTorch.
97 `def __init__(self, in_dim, hidden_dim, num_classes, dropout):`	Constructor: tamaño de entrada (1.024), tamaño interno (128), número de clases y dropout (0,4).
98 `super().__init__()`	Inicializa la clase base.
99 `self.net = nn.Sequential(` 100 `nn.Linear(in_dim, hidden_dim),` 101 `nn.ReLU(),` 102 `nn.Dropout(dropout),` 103 `nn.Linear(hidden_dim, num_classes),` 104 `)`	Arma la red: una capa lineal de 1.024 a 128, activación ReLU, dropout, y una capa lineal final de 128 al número de clases.
106 `def forward(self, x):`	forward: lo que hace con los datos x.
107 `return self.net(x)`	Pasa x por la red y devuelve un puntaje por clase.

29. EarlyStopping

scripts/train_audio_classifier.py:110-126

Detiene el entrenamiento cuando deja de mejorar, para no seguir memorizando los clips de entrenamiento.

110 `class EarlyStopping:`	Define la clase.
112 `def __init__(self, patience=15, min_delta=0.001):`	Constructor: paciencia (cuántas épocas sin mejorar se aguantan, 15) y mejora mínima (0,001).
113 `self.patience = patience` 114 `self.min_delta = min_delta`	Guarda esos dos valores.
115 `self.counter = 0`	Contador de épocas sin mejorar, en 0.
116 `self.best_loss = None`	Mejor pérdida vista hasta ahora (ninguna todavía).
117 `self.early_stop = False`	Bandera que indica si hay que detenerse.
119 `def __call__(self, val_loss):`	Se llama una vez por época con la pérdida de validación.
120 `if self.best_loss is None or val_loss < self.best_loss - self.min_delta:` 121 `self.best_loss = val_loss` 122 `self.counter = 0`	Si es la primera vez o la pérdida mejoró en más de 0,001: guarda esa pérdida como la mejor y reinicia el contador.
123 `else:` 124 `self.counter += 1`	Si no mejoró: suma 1 al contador.
125 `if self.counter >= self.patience:` 126 `self.early_stop = True`	Si el contador llega a la paciencia, activa la bandera de detenerse.

30. Constantes del entrenamiento de audio

scripts/train_audio_classifier.py:19-26

Los valores con los que se entrena el clasificador de audio.

19 `VAL_FRACTION = 0.2`	20 % de los clips se reservan para validar.
20 `SEED = 42`	Semilla 42: hace que el reparto sea siempre el mismo.
21 `EPOCHS = 150`	Máximo 150 épocas.
22 `HIDDEN_DIM = 128`	Tamaño interno de la red: 128.
23 `DROPOUT = 0.4`	Dropout 0,4.
24 `WEIGHT_DECAY = 1e-3`	Decaimiento de pesos: 0,001.
25 `LR = 1e-3`	Tasa de aprendizaje: 0,001.
26 `EARLY_STOPPING_PATIENCE = 15`	Paciencia de la parada temprana: 15 épocas.

31. main (entrenar el modelo de audio)

scripts/train_audio_classifier.py:168-284

Es el programa que entrena el modelo de audio: busca los clips con audio, saca su embedding (o lo toma del caché), entrena la red chica y guarda el mejor resultado en models/audio_classifier.pth.

`168 def main():`	Define la función principal.
`169 clips_por_clase = find_labeled_clips()`	Busca los clips de audio de cada clase.

`170 if not clips_por_clase:`
`171 print(f"No encontré clips con audio en {SESSIONS_DIR}. Grabá primero con "`
`172 "grabar_video_continuo.py.")`
`173 return`
	Si no encuentra ninguno, avisa y termina.

`174 print("Clases encontradas:")`
`175 for clase, wavs in clips_por_clase.items():`
`176 print(f" {clase}: {len(wavs)} clips")`
	Imprime las clases encontradas y cuántos clips tiene cada una.

`178 device = "cuda" if torch.cuda.is_available() else "cpu"`	Elige GPU o CPU.

`179 todos_cacheados = all(`
`180 item.lower().endswith(".npy") for wavs in clips_por_clase.values() for item in wavs`
`181 )`
	Revisa si todos los clips ya tienen su embedding guardado (archivos .npy).

`182 if todos_cacheados:`
`183 print("\nTodos los embeddings ya están cacheados -- no hace falta cargar wav2vec2.")`
`184 model = feature_extractor = None`
	Si todos están guardados, no hace falta cargar wav2vec2.

`185 else:`
`186 print(f"\nCargando wav2vec2 para extraer embeddings (device={device})...")`
`187 model, feature_extractor, _vocab, device = load_model(device=device)`
	Si falta alguno, carga wav2vec2 para calcularlo.

`189 print("\nExtrayendo embeddings (con caché, las corridas siguientes son instantáneas)...")` `190 X, y, class_names = build_embeddings(clips_por_clase, model, feature_extractor, device)`	Construye la matriz X de embeddings, las etiquetas y y los nombres de las clases (con caché, para no recalcular).
`191 print(f"\nTotal: {len(X)} clips, {len(class_names)} clases: {class_names}")`	Imprime el total de clips y clases.
`193 train_idx, val_idx = stratified_split(y, VAL_FRACTION, SEED)` `194 print(f"Train: {len(train_idx)} | Val: {len(val_idx)}")`	Reparte los clips en entrenamiento y validación, manteniendo la proporción de cada clase.
`196 mean = X[train_idx].mean(axis=0, keepdims=True)`	Media de cada una de las 1.024 columnas, solo con los clips de entrenamiento.
`197 std = X[train_idx].std(axis=0, keepdims=True) + 1e-6`	Desviación estándar de cada columna (más un valor diminuto para no dividir por cero).

`199 def to_tensor(idx):`
`200 xs = (X[idx] - mean) / std`
`201 return torch.from_numpy(xs).float().to(device), torch.from_numpy(y[idx]).long().to(device)`
	Función auxiliar to_tensor: normaliza (resta la media, divide entre la desviación) y pasa los datos y las etiquetas a tensores de PyTorch.

`203 x_train, y_train = to_tensor(train_idx)` `204 x_val, y_val = to_tensor(val_idx)`	Aplica esa función a entrenamiento y a validación.
`206 head = AudioClassifierHead(EMBEDDING_DIM, HIDDEN_DIM, len(class_names), DROPOUT).to(device)`	Crea el modelo AudioClassifierHead.
`207 optimizer = torch.optim.Adam(head.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)`	Optimizador Adam.
`208 criterion = nn.CrossEntropyLoss()`	Función de pérdida: entropía cruzada.

`210 history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}`
`211 best_val_acc = -1.0`
`212 best_state = None`
`213 early_stopping = EarlyStopping(patience=EARLY_STOPPING_PATIENCE, min_delta=0.001)`
	Prepara la historia de pérdidas, la mejor precisión (empieza en −1), el espacio para guardar los mejores pesos y la parada temprana.

`for epoch in range(1, EPOCHS + 1):`	Bucle por épocas, de 1 a 150.
`    head.train()`	Pone el modelo en modo entrenamiento.

`    optimizer.zero_grad()`
`    logits = head(x_train)`
`    loss = criterion(logits, y_train)`
`    loss.backward()`
`    optimizer.step()`
	Un paso de entrenamiento: borra gradientes, calcula los puntajes de todos los clips, calcula la pérdida, calcula los gradientes y actualiza los pesos.

`    head.eval()`	Pone el modelo en modo evaluación.

`    with torch.no_grad():`
`        train_acc = (head(x_train).argmax(1) == y_train).float().mean().item()`
`        val_logits = head(x_val)`
`        val_loss = criterion(val_logits, y_val).item()`
`        val_acc = (val_logits.argmax(1) == y_val).float().mean().item()`
	Sin calcular gradientes: mide el acierto en entrenamiento y la pérdida y el acierto en validación.

`    history["train_loss"].append(loss.item())`
`    history["val_loss"].append(val_loss)`
`    history["train_acc"].append(train_acc * 100)`
`    history["val_acc"].append(val_acc * 100)`
	Guarda esos números en la historia.

`    if val_acc > best_val_acc:`
`        best_val_acc = val_acc`
`        best_state = {k: v.clone() for k, v in head.state_dict().items()}`
	Si el acierto de validación es el mejor hasta ahora, guarda una copia de los pesos.

`    if epoch % 10 == 0 or epoch == 1:`
`        print(f"  epoch {epoch:3d} | loss {loss.item():.3f} | val_loss {val_loss:.3f} | "`
`              f"train_acc {train_acc:.2f} | val_acc {val_acc:.2f}")`
	Cada 10 épocas (y en la primera) imprime cómo va.

`    early_stopping(val_loss)`
`    if early_stopping.early_stop:`
`        print(f"  -> Early stopping activado en la epoca {epoch} "`
`              f"(sin mejora en val_loss por {EARLY_STOPPING_PATIENCE} epocas).")`
`        break`
	Informa la pérdida de validación a la parada temprana; si decide parar, lo avisa y sale del bucle.

`plot_training_curves(history)`	Grafica las curvas de pérdida y acierto.
`print(f"\nMejor accuracy de validación: {best_val_acc:.2f} ({int(best_val_acc*len(val_idx))}/{len(val_idx)})")`	Imprime el mejor acierto de validación.

`if len(val_idx) < 10:`
`    print("[AVISO] Set de validación muy chico (<10 clips) -- este número es orientativo, "`
`          "no una medición confiable todavía. Va a mejorar con más datos.")`
	Si hay menos de 10 clips de validación, avisa que el número es solo orientativo.

`head.load_state_dict(best_state)` `head.eval()`	Carga los mejores pesos guardados y pone el modelo en modo evaluación.
`with torch.no_grad():` `    val_pred = head(x_val).argmax(1).cpu().numpy()`	Predice las clases de validación con el mejor modelo.
`val_true = y[val_idx]`	Las etiquetas reales de validación.
`print("\nMatriz de confusión (filas=real, columnas=predicho):")` `print("clases:", class_names)`	Imprime el encabezado de la matriz de confusión.

`n = len(class_names)`
`conf = np.zeros((n, n), dtype=int)`
`for t, p in zip(val_true, val_pred):`
`    conf[t, p] += 1`
	Arma la matriz de confusión: para cada clip suma 1 en (clase real, clase predicha).

`for i, row in enumerate(conf):` `    print(f"  {class_names[i]:35s} {row.tolist()}")`	Imprime la matriz fila por fila.
`plot_confusion_matrix(conf, class_names, best_val_acc)`	Guarda la matriz de confusión como imagen.
`os.makedirs(os.path.dirname(MODEL_OUT), exist_ok=True)`	Crea la carpeta de destino.

272 `torch.save(` 273 `{` 274 `"state_dict": best_state,` 275 `"class_names": class_names,` 276 `"embedding_dim": EMBEDDING_DIM,` 277 `"hidden_dim": HIDDEN_DIM,` 278 `"mean": mean,` 279 `"std": std,` 280 `"val_acc": best_val_acc,` 281 `},` 282 `MODEL_OUT,` 283 `)`	Guarda en models/audio_classifier.pth: los mejores pesos, los nombres de las clases, el tamaño del embedding, el tamaño interno, la media y la desviación (necesarias para normalizar al predecir) y la precisión de validación.
284 `print(f"\nGuardado en {MODEL_OUT}")`	Avisa dónde quedó guardado.

Fase 3 · Análisis de pronunciación (servidor FastAPI)

32. _phoneme_to_token_ids y _greedy_ctc_decode (fonemas ↔ números)

scripts/audio_pronunciation.py:176-195

Dos traductores: uno convierte los fonemas esperados en los números que entiende wav2vec2; el otro convierte lo que oyó el modelo de vuelta en fonemas.

176 `def _phoneme_to_token_ids(fonemas, vocab):`	Define la función: recibe los fonemas y el vocabulario.
177 `    ids = []` 178 `    fonemas_usados = []`	Dos listas vacías: los números y los fonemas que sí se pudieron usar.
179 `    for f in fonemas:` 180 `        if f in vocab:` 181 `            ids.append(vocab[f])` 182 `            fonemas_usados.append(f)`	Por cada fonema, si está en el vocabulario guarda su número y el fonema.
183 `        else:` 184 `            print(f"[AVISO] Fonema '{f}' no está en el vocabulario del modelo, se saltea.")`	Si no está, avisa y lo salta.
185 `    return ids, fonemas_usados`	Devuelve los números y los fonemas usados.
188 `def _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id):`	Define el decodificador: recibe lo que el modelo eligió en cada instante.
189 `    fonemas = []` 190 `    anterior = None`	Lista de fonemas y el fonema anterior.
191 `    for tid in pred_ids:` 192 `        if tid != anterior and tid != blank_id:` 193 `            fonemas.append(id_to_phoneme.get(tid, "?"))` 194 `        anterior = tid`	Por cada instante: si cambió respecto al anterior y no es el símbolo de «silencio», lo agrega; así se quitan repeticiones.
195 `    return fonemas`	Devuelve la lista de fonemas oídos.

33. _distancia_edicion y _align_ops (comparar lo esperado con lo oído)

scripts/audio_pronunciation.py:205-253

Cuentan cuántos cambios hacen falta para convertir una lista en otra, y además dicen cuáles fueron: coincide, cambió, falta o sobra. Es la base para saber qué sonido falló.

205 `def _distancia_edicion(a, b):`	_distancia_edicion: cuántos cambios hacen falta para pasar de a a b.
206 `    n, m = len(a), len(b)` 207 `    dp = [[0] * (m + 1) for _ in range(n + 1)]`	Largos de las dos listas y una tabla para ir anotando los resultados.
208 `    for i in range(n + 1):` 209 `        dp[i][0] = i` 210 `    for j in range(m + 1):` 211 `        dp[0][j] = j`	Pasar de nada a j elementos cuesta j; y al revés.
212 `    for i in range(1, n + 1):` 213 `        for j in range(1, m + 1):` 214 `            if a[i - 1] == b[j - 1]:` 215 `                dp[i][j] = dp[i - 1][j - 1]` 216 `            else:` 217 `                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])`	Llena la tabla: si los elementos son iguales, no cuesta nada; si no, cuesta 1 más que la mejor opción de borrar, insertar o cambiar.
218 `    return dp[n][m]`	El resultado está en la esquina final de la tabla.
221 `def _align_ops(a, b):`	_align_ops: hace lo mismo pero devuelve también cuáles fueron los cambios.

222 n, m = len(a), len(b) 223 dp = [[0] * (m + 1) for _ in range(n + 1)] 224 for i in range(n + 1): 225 dp[i][0] = i 226 for j in range(m + 1): 227 dp[0][j] = j 228 for i in range(1, n + 1): 229 for j in range(1, m + 1): 230 if a[i - 1] == b[j - 1]: 231 dp[i][j] = dp[i - 1][j - 1] 232 else: 233 dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])	Arma la misma tabla de la función anterior.
235 ops = [] 236 i, j = n, m 237 while i > 0 or j > 0:	Parte desde la esquina final de la tabla y camina hacia atrás.
238 if i > 0 and j > 0 and a[i - 1] == b[j - 1]: 239 ops.append(("match", a[i - 1], b[j - 1])) 240 i -= 1 241 j -= 1	Si los elementos son iguales: «match».
242 elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1: 243 ops.append(("sub", a[i - 1], b[j - 1])) 244 i -= 1 245 j -= 1	Si salió de cambiar uno por otro: «sub» (sustitución).
246 elif i > 0 and dp[i][j] == dp[i - 1][j] + 1: 247 ops.append(("del", a[i - 1], None)) 248 i -= 1	Si salió de borrar uno: «del» (omisión).
249 else: 250 ops.append(("ins", None, b[j - 1])) 251 j -= 1	Si no, salió de agregar uno: «ins» (inserción).
252 ops.reverse() 253 return ops	Da vuelta la lista para que quede en orden y la devuelve.

34. diagnose_tipo_error (qué tipo de error fue)

scripts/audio_pronunciation.py:256-293

Con la comparación entre lo esperado y lo oído, pone nombre al error: sustitución típica (por ejemplo r por l), omisión o distorsión en grupos como «bl».

256 _SUSTITUCIONES_CONOCIDAS = { 257 "r": {"l": "lambdacismo", "d": "dentalizacion"}, 258 "ɾ": {"l": "lambdacismo", "d": "dentalizacion"}, 259 "k": {"t": "anteriorizacion", "x": "debilitamiento", "h": "debilitamiento"}, 260 "g": {"d": "anteriorizacion", "x": "debilitamiento", "h": "debilitamiento"}, 261 "s": {"θ": "interdental", "t": "oclusion", "d": "oclusion"}, 262 "ʝ": {"l": "distorsion", "i": "debilitamiento"}, 263 "tʃ": {"ʃ": "distorsion", "t": "debilitamiento", "s": "debilitamiento"}, 264 }	Tabla de errores conocidos: para cada sonido esperado, qué sonido oído lo convierte en qué tipo de error (por ejemplo, una «r» oída como «l» es lambdacismo).
265 _SINFONES = {"bl", "kl", "fl", "gl", "pl"}	Grupos de consonantes con «l» (bl, kl, fl, gl, pl) que se revisan aparte.
268 def diagnose_tipo_error(fonemas_esperados, reconocido_libre_lista): 269 ops = _align_ops(fonemas_esperados, reconocido_libre_lista)	Define la función y alinea lo esperado con lo oído.

271 for idx, (tipo, esperado, reconocido) in enumerate(ops): 272 if esperado == "l" and idx > 0: 273 anterior = ops[idx - 1][1] 274 if anterior and (anterior + "l") in _SINFONES: 275 if tipo == "del": 276 return "omision" 277 if tipo == "sub" and reconocido and reconocido not in ("l",): 278 return "distorsion"	Revisa las «l» de esos grupos: si se borró la «l», es omisión; si se cambió por otro sonido, es distorsión.
279 for idx, (tipo, esperado, reconocido) in enumerate(ops): 280 if tipo == "ins" and idx > 0: 281 previo = ops[idx - 1] 282 if previo[0] in ("match", "sub") and previo[1] and (previo[1] + "l") in _SINFONES: 283 return "epentesis"	Revisa si apareció un sonido de más justo después de la consonante del grupo: es epéntesis.
285 for tipo, esperado, reconocido in ops: 286 tabla = _SUSTITUCIONES_CONOCIDAS.get(esperado) 287 if tabla is None: 288 continue 289 if tipo == "del": 290 return "omision" 291 if tipo == "sub" and reconocido in tabla: 292 return tabla[reconocido]	Para cada sonido, mira la tabla: un borrado es omisión; un cambio conocido devuelve su nombre.
293 return None	Si nada coincide, no hay error con nombre y devuelve None.

35. Constantes del GOP

scripts/audio_pronunciation.py:14-16

Los dos números que deciden cuándo un sonido es «sospechoso» y cuándo una palabra se da por reconocida.

14 GOP_SUSPICIOUS_THRESHOLD = -3.0	Umbral de sospecha: un sonido cuyo puntaje GOP es menor que −3,0 se marca como dudoso. Cuanto más negativo, menos probable fue ese sonido.
16 PALABRA_MISMATCH_THRESHOLD = 0.5	Umbral de reconocimiento: la palabra se da por reconocida si la distancia normalizada entre lo esperado y lo oído es 0,5 o menos.

36. score_pronunciation

scripts/audio_pronunciation.py:296-361

Es el GOP (Goodness of Pronunciation). Alinea los sonidos que debería tener la palabra con el audio real y le pone un puntaje a cada sonido, para saber cuál salió mal y no solo si la palabra falló. También reconoce libremente qué se dijo, sin decirle la palabra.

296 def score_pronunciation(wav_path, palabra, model, feature_extractor, vocab, device):	Define la función: recibe el audio, la palabra esperada, el modelo wav2vec2, su preprocesador, el vocabulario de fonemas y el dispositivo.
297 waveform = load_audio_16k(wav_path)	Carga el audio en 16 kHz y mono.
299 inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")	Prepara la onda en el formato que pide el modelo.
300 with torch.inference_mode(): 301 logits = model(inputs.input_values.to(device)).logits	Pasa el audio por el modelo sin entrenarlo: obtiene los logits, es decir, qué tan probable es cada fonema en cada instante.
302 log_probs = torch.log_softmax(logits, dim=-1)	Los convierte en log-probabilidades (más cerca de 0 = más probable).
304 id_to_phoneme = {v: k for k, v in vocab.items()}	Arma el diccionario inverso: de número a fonema.
305 blank_id = vocab["<pad>"]	Obtiene el identificador del símbolo «blanco» (el silencio o relleno de CTC).

307 `pred_ids = torch.argmax(log_probs, dim=-1)[0].tolist()`	En cada instante elige el fonema más probable.
308 `reconocido_libre = _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id)`	Decodifica esa secuencia quitando repeticiones y blancos: es lo que el modelo «oyó» sin que le dijeran la palabra (reconocimiento libre).
310 `fonemas_esperados = texto_a_fonemas(palabra)`	Convierte la palabra esperada en la lista de fonemas que debería tener.
311 `target_ids, fonemas_usados = _phoneme_to_token_ids(fonemas_esperados, vocab)`	Pasa esos fonemas a los números del vocabulario del modelo y devuelve también cuáles se pudieron usar.
312 `if not target_ids:` 313 `raise ValueError(f"No se pudo mapear ningún fonema de '{palabra}' al vocabulario del modelo.")`	Si ningún fonema de la palabra existe en el vocabulario, lanza un error.
315 `targets = torch.tensor([target_ids], dtype=torch.int32, device=device)`	Convierte los fonemas esperados en un tensor para la alineación.
317 `aligned_labels, aligned_scores = F.forced_align(` 318 `log_probs, targets, blank=blank_id,` 319 `)`	Alineación forzada: acomoda la secuencia esperada sobre los instantes del audio y devuelve, instante por instante, qué fonema le tocó y con qué puntaje.
320 `aligned_labels = aligned_labels[0].tolist()` 321 `aligned_scores = aligned_scores[0].tolist()`	Pasa las etiquetas y los puntajes alineados a listas de Python.
323 `scores_por_fonema = []`	Lista donde se guardará un puntaje por cada fonema.
324 `idx_target = 0`	Índice del fonema esperado que se está procesando.
325 `frame_scores_actual = []`	Puntajes de los instantes que pertenecen al fonema actual.
326 `for label, score in zip(aligned_labels, aligned_scores):`	Recorre cada instante con su etiqueta y su puntaje.
327 `if label == blank_id:` 328 `continue`	Si la etiqueta es el blanco, lo salta.
329 `if idx_target < len(target_ids) and label == target_ids[idx_target]:` 330 `frame_scores_actual.append(score)`	Si la etiqueta es el fonema actual, acumula el puntaje de este instante.
331 `elif frame_scores_actual:` 332 `scores_por_fonema.append(float(np.mean(frame_scores_actual)))` 333 `frame_scores_actual = [score]` 334 `idx_target += 1`	Si empezó un fonema nuevo: cierra el anterior guardando el promedio de sus puntajes, empieza a acumular el nuevo y avanza el índice.
335 `else:` 336 `frame_scores_actual = [score]`	Si es el primer instante con sonido, empieza a acumular.
337 `if frame_scores_actual:` 338 `scores_por_fonema.append(float(np.mean(frame_scores_actual)))`	Al terminar el bucle, cierra el último fonema con su promedio.
340 `while len(scores_por_fonema) < len(fonemas_usados):` 341 `scores_por_fonema.append(None)`	Si faltaron fonemas, los completa con None (sin puntaje).
343 `sospechosos = [` 344 `i for i, s in enumerate(scores_por_fonema)` 345 `if s is not None and s < GOP_SUSPICIOUS_THRESHOLD` 346 `]`	Sospechosos: las posiciones de los fonemas cuyo puntaje es menor que −3,0.
348 `distancia = _distancia_edicion(fonemas_usados, reconocido_libre)`	Calcula la distancia de edición entre los fonemas esperados y lo que el modelo reconoció libremente (cuántos cambios hacen falta para igualarlos).
349 `normalizada = distancia / max(len(fonemas_usados), len(reconocido_libre), 1)`	La normaliza dividiéndola entre la longitud más larga, para que quede entre 0 y 1.
350 `palabra_reconocida = normalizada <= PALABRA_MISMATCH_THRESHOLD`	La palabra se da por reconocida si esa distancia normalizada es 0,5 o menos.

352 return { 353 "fonemas": fonemas_usados, 354 "scores": scores_por_fonema, 355 "sospechosos": sospechosos, 356 "reconocido_libre": " ".join(reconocido_libre), 357 "sonido_reconocido": fonemas_a_texto_aproximado(reconocido_libre), 358 "distancia_normalizada": normalizada, 359 "palabra_reconocida": palabra_reconocida, 360 "tipo_error_audio": diagnose_tipo_error(fonemas_usados, reconocido_libre), 361 }	Devuelve el resultado: los fonemas usados, el puntaje de cada uno, cuáles son sospechosos, lo reconocido libremente en fonemas y en texto aproximado, la distancia normalizada, si la palabra fue reconocida y el tipo de error (omisión, lambdacismo, etc.) que diagnostica diagnose_tipo_error.

37. Constantes del GOP para frases

scripts/audio_pronunciation.py:364-366

Lo mismo que arriba, pero para decidir sobre una palabra dentro de una frase.

364 FRASE_PALABRA_MISMATCH_THRESHOLD = 0.5	La palabra de la frase se da por reconocida si la distancia normalizada es 0,5 o menos.
366 FRASE_COMPLETA_LARGO_MINIMO = 0.6	La frase se da por «dicha completa» si se reconocieron al menos el 60 % de los fonemas que debía tener.

38. score_word_in_sentence

scripts/audio_pronunciation.py:385-498

Hace lo mismo que el GOP anterior, pero con una palabra dentro de una frase: alinea la frase completa, ubica dónde está la palabra objetivo, le pone puntaje solo a esa palabra y además revisa que el resto de la frase esté bien.

385 def score_word_in_sentence(wav_path, frase, palabra_objetivo, model, feature_extractor, vocab, device):	Define la función: recibe el audio, la frase pedida, la palabra objetivo y lo mismo que el GOP de una palabra (modelo, preprocesador, vocabulario, dispositivo).
386 waveform = load_audio_16k(wav_path)	Carga el audio en 16 kHz y mono.
388 inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")	Prepara la onda para el modelo.
389 with torch.inference_mode(): 390 logits = model(inputs.input_values.to(device)).logits	Pasa el audio por el modelo y obtiene los logits.
391 log_probs = torch.log_softmax(logits, dim=-1)	Los convierte en log-probabilidades.
393 id_to_phoneme = {v: k for k, v in vocab.items()} 394 blank_id = vocab["<pad>"]	Diccionario inverso de fonemas y el identificador del blanco.
396 pred_ids = torch.argmax(log_probs, dim=-1)[0].tolist() 397 reconocido_libre = _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id)	Reconocimiento libre: lo que el modelo oyó sin saber la frase.
399 fonemas_frase = texto_a_fonemas(frase)	Fonemas de la frase completa.
400 fonemas_palabra = texto_a_fonemas(palabra_objetivo)	Fonemas solo de la palabra objetivo.
401 target_ids, fonemas_frase_usados = _phoneme_to_token_ids(fonemas_frase, vocab)	Pasa los fonemas de la frase al vocabulario del modelo.
402 if not target_ids: 403 raise ValueError(f"No se pudo mapear ningún fonema de la frase '{frase}' al vocabulario del modelo.")	Si ninguno se puede usar, lanza un error.
405 targets = torch.tensor([target_ids], dtype=torch.int32, device=device)	Tensor con los fonemas esperados de la frase.
406 aligned_labels, aligned_scores = F.forced_align(log_probs, targets, blank=blank_id) 407 aligned_labels = aligned_labels[0].tolist() 408 aligned_scores = aligned_scores[0].tolist()	Alineación forzada de toda la frase y paso a listas.

410 scores_por_fonema = []	Aquí se guardará un puntaje por fonema.
411 frame_ranges = []	Y el rango de instantes (inicio, fin) que ocupa cada fonema, para poder recortar el audio.
412 idx_target = 0 413 frame_scores_actual = [] 414 frame_indices_actual = []	Variables del fonema actual: su índice, sus puntajes y sus instantes.
415 for frame_idx, (label, score) in enumerate(zip(aligned_labels, aligned_scores)):	Recorre cada instante con su número, etiqueta y puntaje.
416 if label == blank_id: 417 continue	Salta los blancos.
418 if idx_target < len(target_ids) and label == target_ids[idx_target]: 419 frame_scores_actual.append(score) 420 frame_indices_actual.append(frame_idx)	Si la etiqueta es el fonema actual, acumula el puntaje y el instante.
421 elif frame_scores_actual: 422 scores_por_fonema.append(float(np.mean(frame_scores_actual))) 423 frame_ranges.append((frame_indices_actual[0], frame_indices_actual[-1])) 424 frame_scores_actual = [score] 425 frame_indices_actual = [frame_idx] 426 idx_target += 1	Si empezó un fonema nuevo, cierra el anterior (promedio de puntajes y rango de instantes), empieza el nuevo y avanza el índice.
427 else: 428 frame_scores_actual = [score] 429 frame_indices_actual = [frame_idx]	Si es el primer sonido, empieza a acumular.
430 if frame_scores_actual: 431 scores_por_fonema.append(float(np.mean(frame_scores_actual))) 432 frame_ranges.append((frame_indices_actual[0], frame_indices_actual[-1]))	Al terminar, cierra el último fonema.
433 while len(scores_por_fonema) < len(fonemas_frase_usados): 434 scores_por_fonema.append(None) 435 frame_ranges.append(None)	Completa con None los fonemas que faltaron.
437 inicio, fin = _localizar_subsecuencia(fonemas_frase_usados, fonemas_palabra)	Busca en qué parte de la frase está la palabra objetivo (posición de inicio y fin).
438 fonemas_usados = fonemas_frase_usados[inicio:fin]	Se queda con los fonemas de esa palabra.
439 scores_recortados = scores_por_fonema[inicio:fin]	Y con sus puntajes.
441 rangos_validos = [r for r in frame_ranges[inicio:fin] if r is not None]	Rangos de instantes de los fonemas de la palabra que sí tienen dato.
442 audio_recortado = None	Aquí se guardará el audio recortado de la palabra.
443 if rangos_validos:	Si hay rangos válidos, calcula el recorte.
444 total_frames = log_probs.shape[1] 445 total_muestras = waveform.shape[0] 446 muestras_por_frame = total_muestras / max(total_frames, 1)	Cuántos instantes y cuántas muestras tiene el audio, y cuántas muestras corresponden a cada instante.
447 frame_inicio = min(r[0] for r in rangos_validos) 448 frame_fin = max(r[1] for r in rangos_validos)	Primer y último instante que ocupa la palabra.
449 colchon_muestras = int(0.15 * TARGET_SAMPLE_RATE)	Un colchón de 0,15 segundos a cada lado, para no cortar la palabra justo.
450 muestra_inicio = max(0, int(frame_inicio * muestras_por_frame) - colchon_muestras) 451 muestra_fin = min(total_muestras, int((frame_fin + 1) * muestras_por_frame) + colchon_muestras)	Convierte esos instantes en muestras y les suma el colchón, sin pasarse de los bordes del audio.
452 if muestra_fin > muestra_inicio: 453 audio_recortado = waveform[muestra_inicio:muestra_fin]	Si el recorte es válido, recorta la onda: ese es el audio de solo la palabra.

455 sospechosos = [456 i for i, s in enumerate(scores_recortados) 457 if s is not None and s < GOP_SUSPICIOUS_THRESHOLD 458]	Sospechosos: los fonemas de la palabra con puntaje menor que −3,0.
460 scores_fuera_de_la_palabra = scores_por_fonema[:inicio] + scores_por_fonema[fin:]	Puntajes de los fonemas que quedan fuera de la palabra (el resto de la frase).
461 sospechosos_fuera = [462 s for s in scores_fuera_de_la_palabra 463 if s is not None and s < GOP_SUSPICIOUS_THRESHOLD 464]	De ellos, los que son sospechosos.
465 resto_de_la_frase_ok = len(sospechosos_fuera) < 2	El resto de la frase se da por bien si hay menos de 2 sospechosos fuera de la palabra.
467 lib_inicio, lib_fin = _localizar_subsecuencia(reconocido_libre, fonemas_palabra)	Busca la palabra dentro del reconocimiento libre de la frase.
468 ventana_reconocida = reconocido_libre[lib_inicio:lib_fin]	Se queda con esa ventana de lo reconocido.
469 distancia = _distancia_edicion(ventana_reconocida, fonemas_palabra)	Distancia de edición entre esa ventana y los fonemas esperados de la palabra.
470 normalizada = distancia / max(len(fonemas_palabra), len(ventana_reconocida), 1)	La normaliza dividiéndola entre la longitud más larga.
471 palabra_reconocida = normalizada <= FRASE_PALABRA_MISMATCH_THRESHOLD	La palabra se da por reconocida si la distancia normalizada es 0,5 o menos.
473 frase_completa_dicha = (474 len(reconocido_libre) >= FRASE_COMPLETA_LARGO_MINIMO * len(fonemas_frase_usados) 475)	La frase se da por dicha completa si lo reconocido tiene al menos el 60 % de la cantidad de fonemas esperados.
477 audio_recortado_path = None	Ruta del audio recortado (todavía no existe).
478 if audio_recortado is not None: 479 import tempfile 480 with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp: 481 audio_recortado_path = tmp.name 482 torchaudio.save(audio_recortado_path, audio_recortado.unsqueeze(0), TARGET_SAMPLE_RATE)	Si hubo recorte, lo guarda en un archivo .wav temporal y anota su ruta.
484 return { 485 "frase": frase, 486 "palabra_objetivo": palabra_objetivo, 487 "fonemas": fonemas_usados, 488 "scores": scores_recortados, 489 "sospechosos": sospechosos, 490 "reconocido_libre": " ".join(reconocido_libre), 491 "sonido_reconocido": fonemas_a_texto_aproximado(ventana_reconocida), 492 "distancia_normalizada": normalizada, 493 "palabra_reconocida": palabra_reconocida, 494 "frase_completa_dicha": frase_completa_dicha, 495 "resto_de_la_frase_ok": resto_de_la_frase_ok, 496 "tipo_error_audio": diagnose_tipo_error(fonemas_palabra, reconocido_libre), 497 "audio_recortado_path": audio_recortado_path, 498 }	Devuelve el resultado: la frase y la palabra, los fonemas y puntajes de la palabra, los sospechosos, lo reconocido libremente, la distancia normalizada, si la palabra fue reconocida, si la frase fue dicha completa, si el resto de la frase está bien, el tipo de error de audio y la ruta del audio recortado.

39. Umbrales de la decisión

server/main.py:57-61

Los cuatro números que deciden cuándo el audio o el video «están seguros».

57 AUDIO_MIN_CONFIDENCE = 0.60	AUDIO_MIN_CONFIDENCE = 0,60: con 60 % de probabilidad o más, el clasificador de audio se considera seguro.

`AUDIO_CONFIRMA_CON_GOP = 0.30`	AUDIO_CONFIRMA_CON_GOP = 0,30: si el análisis de sonidos (GOP) también está de acuerdo, basta un 30 % de seguridad del clasificador.
`ACOUSTIC_MATCH_THRESHOLD = 0.80`	ACOUSTIC_MATCH_THRESHOLD = 0,80: con 80 % o más, el audio «coincide» con una pronunciación correcta.
`VISUAL_MATCH_THRESHOLD = 0.75`	VISUAL_MATCH_THRESHOLD = 0,75: con 75 % o más, el video «coincide» con una clase.

40. _classify_audio

server/main.py:891-919

Usa tu modelo de audio (AudioClassifierHead) para decidir si la palabra sonó correcta. Solo compara contra las clases de la palabra pedida, por ejemplo casa_correcto y casa_incorrecto.

`def _classify_audio(audio_path, palabra):`	Define la función: recibe la ruta del audio y la palabra pedida.
`    clf = _state["audio_clf"]`	Toma el clasificador de audio que se cargó al arrancar el servidor.
`    if clf is None or audio_path is None or not palabra:` `        return None`	Si no hay clasificador, no hay audio o no hay palabra, devuelve None.
`    class_names = _state["audio_clf_classes"]`	Nombres de todas las clases que conoce el clasificador.

`    indices_de_la_palabra = [`
`        i for i, c in enumerate(class_names)`
`        if c == f"{palabra}_correcto" or c.startswith(f"{palabra}_incorrecto")`
`    ]`
	Busca las posiciones de las clases de esa palabra: la correcta y las incorrectas.

`    if not indices_de_la_palabra:` `        return None`	Si la palabra no tiene clases entrenadas, devuelve None.
`    try:`	Empieza un bloque protegido: si algo falla, no se cae todo.
`        emb = extract_embedding(audio_path, _state["audio_model"], _state["audio_feature_extractor"], DEVICE)`	Calcula el embedding del audio con wav2vec2.
`        emb_norm = (emb - _state["audio_clf_mean"]) / _state["audio_clf_std"]`	Lo normaliza con la media y la desviación guardadas al entrenar (las mismas que se usaron entonces).
`        x = torch.from_numpy(emb_norm.reshape(1, -1)).float().to(DEVICE)`	Lo pasa a un tensor de una fila.
`        with torch.no_grad():` `            probs = torch.softmax(clf(x), dim=1)[0]`	Sin calcular gradientes, pasa el embedding por el clasificador; softmax convierte los puntajes en probabilidades que suman 1.
`        probs_familia = probs[indices_de_la_palabra]`	Se queda solo con las probabilidades de las clases de esa palabra.
`        idx_local = int(torch.argmax(probs_familia).item())` `        idx = indices_de_la_palabra[idx_local]`	Toma la clase más probable entre ellas y su posición en la lista completa.
`        clase_predicha = class_names[idx]`	Nombre de la clase predicha.

`        return {`
`            "clase_predicha": clase_predicha,`
`            "prob": float(probs[idx].item()),`
`            "es_correcto": clase_predicha == f"{palabra}_correcto",`
`        }`
	Devuelve la clase predicha, su probabilidad y si es la clase «correcto».

917 except Exception as e: 918 print(f"[AVISO] Falló el clasificador de audio entrenado ({e}) -- sigo solo con el GOP.") 919 return None	Si hubo un error, lo avisa y devuelve None: el sistema sigue solo con el GOP.

41. _run_prediction_pipeline: el video, paso a paso

server/main.py:996-1112

Es la función central del servidor. Esta primera parte procesa el video: lee los frames, saca los puntos de la cara, elige el modelo (cara completa o labios) y obtiene la clase predicha. La parte siguiente (la decisión que combina audio y video) está en su propio bloque.

996 def _run_prediction_pipeline(tmp_path, suffix, landmarker, model, class_names, max_frames, 997 audio_path=None, palabra_esperada=None):	Define la función: recibe el video, el detector, el modelo, los nombres de las clases, el largo fijo, el audio y la palabra pedida.
998 t_start = time.time()	Anota la hora de inicio para medir cuánto tarda.
999 if DEVICE == "cuda": 1000 torch.cuda.empty_cache()	Si hay GPU, libera la memoria que no se usa.
1001 cap = cv2.VideoCapture(tmp_path) 1002 if not cap.isOpened(): 1003 raise HTTPException(status_code=400, detail="No se pudo leer el video enviado.")	Abre el video; si no se puede leer, responde con un error 400.
1005 frames = [] 1006 while True: 1007 ok, frame = cap.read() 1008 if not ok: 1009 break	Lee el video frame por frame hasta que se acaba.
1010 h, w = frame.shape[:2] 1011 if w > MAX_FRAME_WIDTH: 1012 scale = MAX_FRAME_WIDTH / w 1013 frame = cv2.resize(frame, (MAX_FRAME_WIDTH, round(h * scale)), interpolation=cv2.INTER_AREA)	Si el frame es más ancho que 640 píxeles lo achica, para ir más rápido.
1014 frames.append(frame)	Guarda el frame.
1015 cap.release() 1016 print(f"[predict] frames leídos: {len(frames)} ({time.time() - t_start:.1f}s)")	Cierra el video y avisa cuántos frames leyó y en cuánto tiempo.
1018 total = len(frames) 1019 if total == 0: 1020 return { 1021 "valid": False, 1022 "reason": "no se pudo leer ningún frame del video -- el formato/códec " 1023 "puede no ser compatible con el servidor (probá grabar en H.264/mp4)", 1024 }	Si no se pudo leer ningún frame, responde que la toma no es válida y explica por qué.
1025 if total < MIN_FRAMES_VALID: 1026 return { 1027 "valid": False, 1028 "reason": f"toma muy corta ({total} frames, minimo {MIN_FRAMES_VALID}) " 1029 "-- probablemente no se dijo la frase completa", 1030 }	Si hay menos de 15 frames, responde que la toma es muy corta.
1032 landmarks_por_frame = detect_landmarks_batch(landmarker, frames) 1033 print(f"[predict] landmarks detectados ({time.time() - t_start:.1f}s)")	Detecta los 68 puntos de la cara en todos los frames.
1035 positions = [] 1036 detected = 0 1037 last_valid_vector = None	Prepara las listas para el modelo de labios y los contadores.
1038 for landmarks_px in landmarks_por_frame: 1039 vector = normalize_lip_landmarks(landmarks_px) if landmarks_px is not None else None	Por cada frame, normaliza los puntos de la boca.

1040 if vector is not None and is_outlier(vector, last_valid_vector): 1041 vector = None	Si el vector salta demasiado respecto al anterior (error del detector), lo descarta.
1042 if vector is not None: 1043 detected += 1 1044 last_valid_vector = vector 1045 positions.append(vector)	Cuenta los frames buenos, recuerda el último bueno y guarda el vector (o None).
1047 detection_rate = detected / total if total else 0 1048 filled = fill_missing_frames(positions)	Calcula en qué porcentaje de frames se detectó la cara y rellena los frames sin detección.
1049 if filled is None or detection_rate < MIN_DETECTION_RATE_VALID: 1050 return { 1051 "valid": False, 1052 "reason": f"cara detectada solo en {detected}/{total} frames " 1053 f"({detection_rate*100:.0f}%) -- encuadre malo, repetí la toma", 1054 }	Si casi no se detectó la cara (menos de 60 %), responde que el encuadre fue malo y que repita.
1056 modelo_video = "labios" 1057 clases_cara = _state["class_names_cara"]	Por defecto usa el modelo de labios; mira qué clases conoce el modelo de cara completa.
1058 if (_state["model_cara"] is not None and palabra_esperada is not None 1059 and f"{palabra_esperada}_correcto" in clases_cara and f"{palabra_esperada}_incorrecto" in clases_cara):	Solo prueba con cara completa si el modelo existe, se pidió una palabra y ese modelo tiene las dos clases de la palabra (correcta e incorrecta).
1060 posiciones_cara = [] 1061 detectados_cara = 0 1062 ultimo_cara = None	Prepara las listas para la versión de cara completa.
1063 for landmarks_px in landmarks_por_frame: 1064 vector_cara = normalize_face_landmarks(landmarks_px) if landmarks_px is not None else None 1065 if vector_cara is not None and is_outlier(vector_cara, ultimo_cara): 1066 vector_cara = None 1067 if vector_cara is not None: 1068 detectados_cara += 1 1069 ultimo_cara = vector_cara 1070 posiciones_cara.append(vector_cara)	Repite el mismo recorrido, pero normalizando los 68 puntos con los ojos.
1071 tasa_cara = detectados_cara / total 1072 relleno_cara = fill_missing_frames(posiciones_cara)	Calcula el porcentaje de frames con cara entera y rellena los vacíos.
1073 if relleno_cara is not None and tasa_cara >= MIN_DETECCION_CARA: 1074 filled = relleno_cara 1075 model = _state["model_cara"] 1076 class_names = clases_cara 1077 max_frames = _state["max_frames_cara"] 1078 modelo_video = "cara_completa"	Si la cara se vio en el 60 % o más de los frames, cambia a esos datos y a ese modelo, y anota que usó «cara_completa».
1079 else: 1080 print(f"[predict] cara completa no usable ({detectados_cara}/{total} frames con la cara entera): uso el modelo de labios")	Si no, avisa y sigue con el modelo de labios.
1081 sequence, mask = preprocess_sequence(filled, max_frames) 1082 sequence, mask = sequence.to(DEVICE), mask.to(DEVICE)	Deja la secuencia lista (60 frames, con su máscara) y la manda a la GPU.
1083 with torch.inference_mode(): 1084 logits = model(sequence, src_key_padding_mask=mask) 1085 probs = torch.softmax(logits, dim=1)[0]	Sin calcular gradientes, pasa la secuencia por el modelo (TCN + Conformer) y convierte los puntajes en probabilidades.
1086 print(f"[predict] modelo Conformer ({modelo_video}) listo ({time.time() - t_start:.1f}s total)")	Avisa qué modelo corrió y cuánto tardó.

1088 sorted_probs, sorted_idx = torch.sort(probs, descending=True) 1089 top_idx = int(sorted_idx[0].item()) 1090 top_prob = float(sorted_probs[0].item()) 1091 second_prob = float(sorted_probs[1].item()) if len(sorted_probs) > 1 else 0.0 1092 margin = top_prob - second_prob	Ordena las clases de más a menos probable, toma la primera, la segunda y el margen entre las dos.
1093 clase_predicha = class_names[top_idx]	Nombre de la clase ganadora, por ejemplo casa_correcto.
1094 print(f"[predict] video[{modelo_video}]: clase={clase_predicha} prob={top_prob:.2f} margin={margin:.2f} " 1095 f"(2da opcion: {class_names[int(sorted_idx[1].item())] if len(sorted_idx) > 1 else '?'} " 1096 f"{second_prob:.2f}) | esperado={palabra_esperada}")	Imprime en la consola lo que predijo y la segunda opción (sirve para depurar).
1097 palabra, pronunciacion_correcta, tipo_error = _parse_clase_predicha(clase_predicha)	Separa el nombre de la clase en la palabra, si fue correcta y qué tipo de error.
1099 def _palabra_base(nombre): 1100 return nombre.split("_")[0]	Función corta que saca la palabra base (antes del primer guion bajo).
1102 palabra_distinta = False 1103 if palabra_esperada is not None and palabra != palabra_esperada: 1104 pidio_palabra_completa = palabra_esperada == _palabra_base(palabra_esperada) 1105 misma_palabra_base = _palabra_base(palabra) == _palabra_base(palabra_esperada) 1106 if not (pidio_palabra_completa and misma_palabra_base): 1107 pronunciacion_correcta = False 1108 tipo_error = None 1109 palabra_distinta = True	Si el modelo vio otra palabra distinta de la pedida, lo toma como incorrecto; salvo que se pidió la palabra completa y es la misma palabra base (por ejemplo casa y casa_correcto).
1111 all_probs = {class_names[i]: float(probs[i].item()) for i in range(len(class_names))}	Guarda la probabilidad de todas las clases.

42. La decisión final: audio + video (dentro de _run_prediction_pipeline)

server/main.py:1113-1183

Es el corazón del servidor: junta lo que dijo el video con lo que dijo el audio y decide el veredicto. Regla principal: el audio manda cuando está seguro; el video solo bloquea si está muy seguro de que se dijo la misma palabra mal.

1113 es_palabra_completa = (palabra_esperada or palabra) and "_" not in (palabra_esperada or palabra)	Decide si se pidió una palabra completa (sin guion bajo, por ejemplo «casa» y no «casa_correcto»).
1114 audio_resultado = (1115 _score_audio_pronunciation(audio_path, palabra_esperada or palabra) 1116 if es_palabra_completa else None 1117)	Si es palabra completa, calcula el GOP del audio; si no, lo deja vacío.
1118 audio_ok = (1119 audio_resultado is not None 1120 and len(audio_resultado["sospechosos"]) == 0 1121 and audio_resultado["palabra_reconocida"] 1122)	audio_ok: el GOP no marcó sonidos sospechosos y la palabra se reconoció.
1124 audio_clf_resultado = _classify_audio(audio_path, palabra_esperada or palabra)	Calcula la decisión del clasificador de audio entrenado.
1125 prob_str = f"{audio_clf_resultado['prob']:.2f}" if audio_clf_resultado else "?" 1126 print(f"[predict] audio GOP {'OK' if audio_resultado is None else ('bien' if audio_ok else 'sospechoso')} | " 1127 f"clasificador {'sin entrenar' if audio_clf_resultado is None else audio_clf_resultado['clase_predicha']} " 1128 f"prob={prob_str} (umbral {AUDIO_MIN_CONFIDENCE}) video_prob={top_prob:.2f} " 1129 f"({time.time() - t_start:.1f}s total)")	Imprime en la consola una línea con lo que dijo el GOP, el clasificador y la probabilidad del video (útil para depurar).

1131 acoustic_match = (1132 audio_clf_resultado is not None 1133 and audio_clf_resultado["prob"] >= ACOUSTIC_MATCH_THRESHOLD 1134 and audio_clf_resultado["es_correcto"] 1135)	acoustic_match: el clasificador existe, está seguro (80 % o más) y dice «correcto».
1136 visual_match = top_prob >= VISUAL_MATCH_THRESHOLD and pronunciacion_correcta	visual_match: el video está seguro (75 % o más) y su clase es una pronunciación correcta.
1138 confianza_suficiente = audio_clf_resultado is not None	Hay confianza suficiente solo si existe el resultado del clasificador de audio.
1140 audio_gop_ok = (1141 audio_resultado is not None 1142 and not audio_resultado["sospechosos"] 1143 and audio_resultado["palabra_reconocida"] 1144)	audio_gop_ok: el GOP no marcó sospechosos y la palabra se reconoció.
1145 audio_confirma = (1146 audio_clf_resultado is not None 1147 and audio_clf_resultado["es_correcto"] 1148 and (audio_clf_resultado["prob"] >= AUDIO_MIN_CONFIDENCE 1149 or (audio_gop_ok and audio_clf_resultado["prob"] >= AUDIO_CONFIRMA_CON_GOP)) 1150)	audio_confirma: el clasificador dice «correcto» y está seguro (60 % o más), o está medianamente seguro (30 % o más) y el GOP está de acuerdo.
1151 video_dice_mal_misma_palabra = (1152 (not palabra_distinta) and top_prob >= VISUAL_MATCH_THRESHOLD and not pronunciacion_correcta 1153)	El video está seguro (75 % o más) de que se dijo la MISMA palabra pero mal pronunciada.
1155 if not confianza_suficiente: 1156 veredicto_final = "no_concluyente" 1157 tipo_error = None	Si no hay clasificador de audio, el veredicto es «no concluyente».
1158 elif acoustic_match and visual_match: 1159 veredicto_final = "bien" 1160 tipo_error = None	Si audio y video coinciden en que está bien, el veredicto es «bien».
1161 elif audio_confirma and not video_dice_mal_misma_palabra: 1162 veredicto_final = "bien" 1163 tipo_error = None 1164 print(f"[predict] audio confirma '{palabra_esperada}' (clasificador {audio_clf_resultado['prob']:.2f}, " 1165 f"GOP {'bien' if audio_gop_ok else 'dudoso'}); el video ({palabra}, {top_prob:.2f}) no lo contradice " 1166 f"con seguridad -> bien")	Si el audio confirma y el video no contradice con seguridad, el veredicto es «bien» (y lo anota en la consola).
1167 elif (not acoustic_match) and visual_match: 1168 veredicto_final = "a_practicar" 1169 tipo_error = "sustitucion_acustica"	Si el audio no coincide pero el video sí, «a practicar» con error de sustitución acústica.
1170 elif (not acoustic_match) and (not visual_match): 1171 veredicto_final = "a_practicar" 1172 tipo_error = "motor_severo"	Si ni el audio ni el video coinciden, «a practicar» con error motor severo.
1173 else: 1174 veredicto_final = "a_practicar" 1175 tipo_error = "compensacion_visual"	En cualquier otro caso (el audio confirma pero el video está seguro de que está mal), «a practicar» con compensación visual.
1177 capture_file = None	Aún no se guardó ningún clip.
1178 if confianza_suficiente: 1179 capture_file = _save_practice_clip(clase_predicha, tmp_path, suffix)	Si hay confianza suficiente, guarda el clip del intento.
1180 db.save_attempt(1181 class_name=clase_predicha, palabra=palabra, correcta=(veredicto_final == "bien"), 1182 tipo_error=tipo_error if veredicto_final == "a_practicar" else None, prob=top_prob, 1183)	Y registra el intento en la base de datos: clase, palabra, si fue correcta, tipo de error y probabilidad.

43. predict (el endpoint /predict)

server/main.py:1212-1262

Es la puerta que usa la app: recibe el video, el audio y la palabra, y llama al pipeline correspondiente.

```
@app.post("/predict")
async def predict(
    file: UploadFile = File(None),
    audio: UploadFile = File(None),
    palabra: str = Form(None),
):
```

Define la ruta POST /predict: recibe el video (opcional), el audio (opcional) y la palabra pedida.

```
    if _state["model"] is None:
        raise HTTPException(status_code=503, detail="El servidor todavía está cargando el modelo.")
```

Si el modelo todavía se está cargando, responde 503.

```
    audio_path = None
    if audio is not None:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp_audio:
            shutil.copyfileobj(audio.file, tmp_audio)
            audio_path = tmp_audio.name
```

Si llegó audio, lo guarda en un archivo temporal.

```
    if file is None:
        if audio_path is None:
            raise HTTPException(status_code=400, detail="Hace falta mandar audio o video.")
```

Si no llegó video y tampoco audio, responde que falta algo.

```
        try:
            return _run_audio_only_pipeline(audio_path, palabra_esperada=palabra)
        finally:
            try:
                os.remove(audio_path)
            except OSError:
                pass
```

Si solo hay audio, corre el pipeline solo de audio y borra el archivo temporal.

```
    landmarker = _state["landmarker"]
    model = _state["model"]
    class_names = _state["class_names"]
    max_frames = _state["max_frames"]
```

Toma los modelos que quedaron cargados al arrancar.

```
    suffix = os.path.splitext(file.filename or "")[1] or ".mp4"
    with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
        shutil.copyfileobj(file.file, tmp)
        tmp_path = tmp.name
```

Guarda el video en un archivo temporal.

```
    try:
        return _run_prediction_pipeline(
            tmp_path, suffix, landmarker, model, class_names, max_frames,
            audio_path=audio_path, palabra_esperada=palabra,
        )
```

Corre el pipeline completo con el video, el audio y la palabra.

```
    finally:
        try:
            os.remove(tmp_path)
        except OSError:
            pass
```

Al final, borra el video temporal.

```
        if audio_path is not None:
            try:
                os.remove(audio_path)
            except OSError:
                pass
```

Y también el audio temporal.